

Swarm Failover

A drone swarm that keeps flying its mission when the leader fails - simulation, tests, results, how to run everything and what is left, in one file.

Generated 27 September 2026 from the repository at `/home/saad-rafique/Documents/drone_swarm`.
Everything described here was built and tested in simulation; no real drone has flown this code.
Source documents: README.md, docs/*.md, reports/*.md. Regenerate with `scripts/make_handover_pdf.py`.

Contents

1. [Start here](#)
2. [How it works](#)
3. [Results](#)
4. [Key figures](#)
5. [How to install and run everything](#)
6. [Decisions and why](#)
7. [Known issues, what is not done, next steps](#)
8. [Long RL training on Kaggle](#)
9. [Appendix A - Phase 0 report](#)
10. [Appendix B - Phase 1 report](#)
11. [Appendix C - Phase 2 report](#)
12. [Appendix D - Phase 3 report](#)
13. [Appendix E - Phase 4 report \(not completed\)](#)
14. [Appendix F - Scaling test](#)
15. [Appendix G - Project specification](#)

Start here

README.MD

Swarm Failover

A drone swarm that keeps flying its mission when the leader fails, built and tested entirely in simulation on one laptop. Every drone runs the same agent code: it elects a leader (lowest healthy ID, with a term counter), holds its slot in a V formation, keeps 5 m from other drones, stays inside a geofence, and takes over within about two seconds when the leader crashes, loses its radio or runs low on battery. On top of that: obstacle avoidance among real buildings (OpenStreetMap), a learned (reinforcement learning) avoider compared fairly with a classical one, city-to-city routes with charging stops, and a ground-control web app with a 2-D map, a 3-D view, fault buttons, results and these documents.



In short. A group of drones (a swarm) that keeps flying its mission when the leader crashes, loses its radio or runs low on battery: the next drone takes over in about 2 seconds. Everything was built and tested in simulation on a laptop; no real drone has flown this code. To start, double-click **Swarm Control** in the project folder (or the **Swarm Control** icon on the desktop): the whole program starts and opens in the browser. Long RL training on Kaggle, step by step: docs/KAGGLE.md (or docs/KAGGLE_GUIDE.pdf). Everything in one file: docs/HANDOVER.pdf.

Scope and safety. Simulation only: no code here opens a serial port, talks to a real flight controller or flashes firmware. Payloads, targeting and anything that engages objects are out of scope (see docs/SPECIFICATION.md).

Status (27 September 2026)

Part	State	Evidence
Phase 0 — environment, one PX4 drone	passed	reports/PHASE_0.md
Phase 1 — 10 PX4 drones on this laptop	passed	reports/PHASE_1.md
Phase 2 — agent logic, 1,000 random fault runs	passed	reports/PHASE_2.md
Phase 3 — 10-drone PX4 formation flights	passed	reports/PHASE_3.md
Swarm size 1–100 drones (fast simulator)	passed	reports/SCALING.md

Part	State	Evidence
Obstacle avoidance, RL vs classical	first results	docs/RESULTS.md, reports/logs/rl/
City-to-city routes, charging stops	working in the fast simulator; 12 km route completed end to end	docs/RESULTS.md, reports/logs/long_route/
Islamabad → Lahore run (272.55 km, 65 charging stops)	stopped on purpose after 76 km: 18 stops used, 10/10 drones flying, 0 hits	docs/RESULTS.md, reports/logs/long_route/
Ground-control app (2-D map, 3-D view, faults, results, docs)	working	docs/RUNBOOK.md
Phase 4 — fault trials on PX4	started, stopped by the owner (laptop too weak; indicative results and a full how-to)	reports/PHASE_4.md
Phase 5 — radio realism sweep	not run	docs/KNOWN_ISSUES.md
Phase 6 — mixed reality, flight test plan	not done	docs/KNOWN_ISSUES.md
Longer RL training on Kaggle	kit ready and checked, not run (step-by-step guide)	docs/KAGGLE.md
Real drones	never flown	—

Tests: `python3 -m pytest` → 130 passed, 4 skipped (the skipped ones need ROS 2 or PyTorch); with PyTorch (`PYTHONPATH=src .venv/bin/python -m pytest`) 133 passed, 2 skipped.

Quick start

One click, any of these (each starts the ground-control app if it is not running and opens `http://localhost:8080` in the browser):

- in this project folder: double-click **Swarm Control** (a small program built by `bash scripts/install_launcher.sh`; it is not in git, so a fresh copy needs that command once), or **Start Swarm Control.sh** (if the Files app opens it as text: right-click → Run as a Program);
- the **Swarm Control** icon on the desktop or in the application menu (also installed by `bash scripts/install_launcher.sh`).

Stop the app with `scripts/stop_swarm.sh`.

From a terminal:

```
python3 scripts/gcs.py          # then open http://localhost:8080
```

Pages: **Mission** (2-D map, mission setup, fault buttons), **3D view** (3-D drones over terrain and 3-D buildings, chase / orbit / top cameras, fault buttons), **Results**, **PX4 flights** (replay of the PX4 runs), **Docs** (these documents and the handover PDF). Satellite maps and the 3-D view need your own Mapbox and Cesium ion tokens in `config/map_keys.local.yaml` (copy of `config/map_keys.example.yaml`; git-ignored).

Everything else (PX4 flights, tests, RL training, evaluations): `docs/RUNBOOK.md`.

Repository map

Path	What is there
<code>src/swarm_agent/</code>	The agent that would run on every drone: election, formation, safety, geometry, heartbeat codec, obstacles, route planner, avoiders, ROS 2 node. Pure Python + numpy (ROS only in <code>ros_node.py</code>).
<code>src/swarm_tools/</code>	Simulation and test tools: PX4/MAVROS launcher, link emulator, logger, fast simulator (<code>puressim.py</code>), obstacle simulator for RL (<code>obstacle_sim.py</code> , <code>torch_sim.py</code>), OpenStreetMap loader, ground-control app (<code>gcs/</code>).
<code>config/</code>	<code>swarm.yaml</code> (every tunable), MAVROS plugin list, map-key template.
<code>scripts/</code>	Entry points: flights, tests, evaluations, training, plots, launcher, PDF builder.
<code>tests/</code>	135 tests (pytest; some need PyTorch or ROS 2 and are skipped without them).
<code>reports/</code>	Phase reports, plots and raw logs (<code>reports/logs/</code>).
<code>models/avoid_policy.npz</code>	The trained obstacle-avoidance policy (numpy weights).
<code>data/osm/</code>	Cached OpenStreetMap downloads ((c) OpenStreetMap contributors, ODbL).
<code>kaggle/</code>	Notebook for long RL training on Kaggle (the code zip is built by <code>scripts/make_kaggle_bundle.py</code>).
<code>Swarm Control</code> , <code>Start Swarm Control.sh</code>	One-click launchers for the app (see Quick start).
<code>docs/</code>	Architecture, runbook, results, decisions, known issues, Kaggle, handover PDF.
<code>docs/SPECIFICATION.md</code>	The original project brief: goals, hard rules, phases and acceptance criteria.

Documents

- `docs/HANDOVER.pdf` — everything below in one file, for handing the project over.
- `docs/ARCHITECTURE.md` — how it works.
- `docs/RUNBOOK.md` — how to install and run every part.
- `docs/RESULTS.md` — all results with numbers and where the evidence is.
- `docs/DECISIONS.md` — the important design decisions and why.
- `docs/KNOWN_ISSUES.md` — limitations, what is not done, next steps, road to real drones.
- `docs/KAGGLE.md` — long RL training on Kaggle, step by step: which files, how to start it, what gets trained, where to paste the results, how to change things; also as `docs/KAGGLE_GUIDE.pdf`.

Main versions

Ubuntu 24.04.5, ROS 2 Jazzy, PX4 v1.18.0-rc1 (fca3df865a , target px4_sitl_sih), MAVROS 2.15.1, Python 3.12.3, numpy 1.26.4, scipy 1.11.4; RL in .venv: PyTorch 2.14.0 (CPU), Stable-Baselines3 2.9.0, Gymnasium 1.3.0. Laptop: Intel i3-1115G4 (2 cores, 4 threads), 8 GB RAM, no GPU.

Credits

Map data (c) OpenStreetMap contributors (ODbL). Satellite imagery (c) Mapbox, (c) Maxar, or Esri World Imagery. 3-D terrain, buildings and photorealistic tiles through Cesium ion. Every result above comes from logged runs in `reports/`.

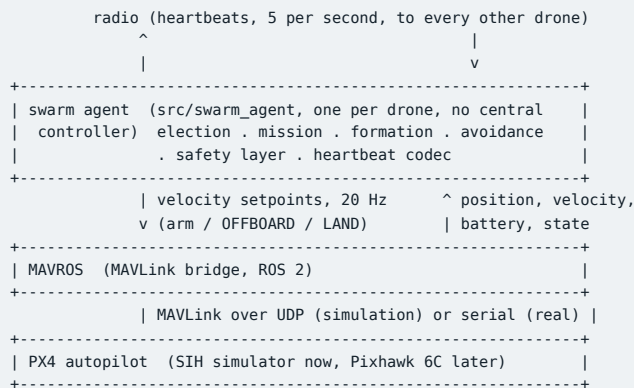
How it works

DOCS/ARCHITECTURE.MD

How it works

This document explains the system from the drone outwards. File names point at the code; every tunable number lives in `config/swarm.yaml`.

1. One drone



- The agent logic is plain Python (`agent_core.py` and the modules it uses). The same class runs inside the ROS 2 node (`ros_node.py`, one per drone in namespace `/uavN`) and inside the fast simulator (`swarm_tools/puresim.py`).
- MAVROS runs with a lean plugin list (`config/mavros_pluginlists.yaml`: `sys_status`, `sys_time`, `command`, `local_position`, `global_position`, `setpoint_velocity`) and per-namespace remaps of `/tf`, `/tf_static` and `/parameter_events`; the default plugin set made CPU grow with the square of the drone count (Phase 1).
- Moving a drone to real hardware changes its MAVROS connection URL (UDP -> serial) and replaces the radio emulator by a real radio link. The agent code stays the same.

2. Positions: one shared frame

Every drone converts its own latitude/longitude/altitude to one shared East-North-Up frame anchored at `config.origin` (WGS-84 -> ECEF -> ENU, `geometry.EnuFrame`). Drones never compare their MAVROS local frames, which each start at their own power-on point. On long routes the ground falls below the tangent plane (about 5.7 km at 270 km), so positions are converted back with `EnuFrame.surface_point`, which puts them on the ground (without it they would be ~240 m off).

3. Heartbeat

Each drone broadcasts a heartbeat 5 times a second (`heartbeat.py`, codec version 2): id, role, term, mission phase, leader id, handover target, flags (MASTER_OK, READY, ELIGIBLE, ORPHAN, AIRBORNE), battery, position, velocity, formation heading, time stamp, and - from the leader - the list of members it hears as a bit mask. A fixed 43-byte header plus only as many mask bytes as the highest ID needs: 45 bytes for 10 drones, 56 for 100, up to ID 250 (PX4's `MAV_SYS_ID` limit). In simulation the heartbeats go through `link_emulator.py`, which can add delay, jitter, loss, cut a drone's transmitter or split the swarm into groups.

4. Choosing the leader (`election.py`)

- The lowest-numbered alive and eligible drone leads. Eligible: position valid, battery above the handover level (30 %), not retired.
- Every leader claim increases a `term` number. A leader that hears another leader with a higher term (or the same term and a lower ID) steps down, so after a radio split heals there is exactly one leader again.
- If nobody hears the leader for 1.5 s, the next eligible drone claims - but only if no other drone still hears a leader (MASTER_OK flag) and no lower eligible ID is alive. No pre-emption: a recovered lower ID does not take the job back.
- At startup drones listen first and the first election waits until the whole fleet is heard (or a timeout), so the intended leader is not beaten by a drone that booted earlier.
- A leader at 30 % battery names a successor (planned handover), leaves the formation and flies home.
- While no leader is known, drones hover.

5. Mission (`agent_core.py`)

The leader runs the mission and every follower copies its phase from the heartbeat: IDLE -> TAKEOFF (whole fleet ready) -> CRUISE -> HOLD (10 s at the goal) -> LAND. On long routes: at each charging stop LAND -> CHARGE (on the ground until every battery is ~95 %) -> TAKEOFF (vertically where each drone stands) -> CRUISE again. In open sky the leader flies a straight leg to the goal (fixed heading plus a cross-track correction) with a speed ramp; with obstacles it follows the planned route (pure pursuit), turning the formation heading at most 20 degrees per second.

6. Formation (`formation.py`)

- V formation, 10 m between neighbours, 45 degree arms. Even IDs fly the left arm, odd IDs the right, in ID order ("parity arms"), so losing a drone only slides the drones behind it on the same arm.
- Follower command = leader velocity (feed-forward) + P-control toward its slot, saturated at 3 m/s.

- The leader's heartbeat lists its members; a drone the leader cannot hear is an "orphan" and flies 8 m higher behind its arm. Moves longer than 12 m happen 6 m below the formation ("transit layer") so crossing paths are vertically separated.

7. Safety layer (every command, every drone)

- Repulsion from any drone closer than 7.5 m (1.5 x the 5 m minimum), including drones that just went silent ("ghosts", extrapolated for 3 s).
- Geofence: no motion outward beyond radius - margin, capped altitude.
- Emergencies: GPS lost or battery at 10 % -> hand over, drop below the formation while keeping its speed (so the drones behind pass over it), land where it is. On long routes a low battery also lands in place (home may be far behind).

8. Obstacles and avoidance

- `obstacles.py`: 2-D obstacle map at flight height: building footprints (polygons) and trees (circles) with a grid index; 24-ray range "sensor", nearest obstacle point, clearance, occupancy.
- `planner.py`: the leader's route by A* on an inflated grid (8 m clearance) with line-of-sight shortcuts; long routes chunk by chunk; charging stops in open spots about every half battery.
- `avoidance.py`: followers add a horizontal correction from an avoider, then the safety layer:
- none;
- classical: potential field (repulsion from the nearest obstacle point, sliding, a tangential push) plus a stopping-distance brake, tuned on training scenarios;
- learned: a small neural network (2 x 128, tanh) trained with PPO, run with numpy only;
- learned + brake: the network's command passed through the same brake. All avoiders see the same input: desired velocity, own velocity, slot error, 24 range readings, the nearest obstacle point, the three nearest drones. The avoider runs at 10 Hz.
- Map data: `swarm_tools/osm.py` downloads buildings, woods and trees from OpenStreetMap (Overpass API) and caches them in `data/osm/`. Flight height decides what is an obstacle: low (16 m) - every building and wood; normal (30 m) - only structures of 25 m or more (OSM height or storeys).

9. Simulators

Simulator	What it runs	Used for
PX4 SIH + MAVROS (<code>swarm_tools/sim_launch.py</code>)	the real autopilot firmware, simple physics	Phases 0-3: real flights of up to 10 drones
Fast simulator (<code>swarm_tools/puresim.py</code>)	the real agent code, point-mass physics, simulated radio	1,000-run tests, scaling, the ground-control app
Obstacle simulator (<code>swarm_tools/obstacle_sim.py</code>)	numpy formation-through-obstacles	RL training (CPU) and the fair comparison
Batched simulator (<code>swarm_tools/torch_sim.py</code>)	the obstacle simulator on a GPU, 1024 missions at once	long RL training on Kaggle

The obstacle simulator re-implements the follower law and repulsion in numpy; tests check it against the agent code (`tests/test_avoidance.py`), and the batched version is checked step by step against the numpy one (`tests/test_torch_sim.py`).

10. Ground-control app (`swarm_tools/gcs/`)

`fastsim_backend.py` runs the fast simulator in a background thread (1x to Max) and turns its state into latitude/longitude; `server.py` (standard library HTTP server on 127.0.0.1) streams a snapshot every 100 ms (Server-Sent Events) and accepts commands (start, pause, speed, faults, radio split). Pages in `static/`: Mission (Leaflet map), 3D view (CesiumJS), Results, Docs. Slow preparation (map downloads, long-route planning) runs in a background thread with progress on the page.

- **3D view** (`static/3d.html` , `static/view3d.js`): every drone is a small 3-D quadcopter (`static/drone.glb` , generated by `scripts/make_drone_model.py` - no outside model files) whose white body is tinted by role. Snapshots arrive every 100 ms; each drone glides from where it is drawn to its newest position over one snapshot interval, so the motion is smooth (about 0.1 s behind the simulation). The simulators have flat ground, so each drone is drawn at its simulated height above the terrain under it; a drone appears only once that terrain height is known. V lines join the leader to the followers behind it on each side. In the Chase, Orbit and Top modes the camera is placed every frame around the leader and the followers in its formation, and lifts itself when it would go below the terrain.
- **Takeover timing**: when a fault hits the leader (a fault button, a crash, an empty battery), the backend remembers when; the next "became master" event says how many seconds later the new leader took over (for example 1.5 s after a crash, 0.1 s after a planned handover).
- **Without the browser**: `scripts/run_route.py` drives the same backend step by step and writes a JSON record (events, progress, formation error and closest pair while cruising); the long-route evidence in `reports/logs/long_route/` comes from it.
- **One-click start**: `scripts/start_swarm.sh` starts `scripts/gcs.py` in the background if it is not already running and opens the browser. The **Swarm Control** program in the project folder (`scripts/launcher.c`), `Start Swarm Control.sh` and the desktop icon all call it.

11. Where to change things

To change	Edit
spacing, speeds, gains, timeouts, battery levels, geofence	<code>config/swarm.yaml</code>
election rules	<code>src/swarm_agent/election.py</code> (+ <code>tests/test_election.py</code>)
formation shape	<code>src/swarm_agent/formation.py</code> (V only today)
what counts as an obstacle, flight heights	<code>src/swarm_tools/osm.py</code> , <code>gcs/fastsim_backend.py</code>
the avoider used by the app	<code>models/avoid_policy.npz</code> (RL) or <code>PotentialField</code> parameters
the 3-D view (drone look, cameras, colours)	<code>src/swarm_tools/gcs/static/view3d.js</code> , <code>3d.html</code> ; the model: <code>scripts/make_drone_model.py</code>
reward / training	<code>src/swarm_tools/obstacle_sim.py</code> , <code>scripts/rl_train.py</code> , <code>scripts/rl_train_gpu.py</code>

Results

DOCS/RESULTS.MD

Results

Everything here is simulation. Each number points at the file it came from.

PX4 autopilot in simulation (SIH + MAVROS, this laptop)

What	Result	Evidence
One drone: arm, climb to 10 m, hover 20 s, land	passed twice; 135 MB RAM for PX4 + MAVROS	reports/PHASE_0.md
10 drones: all arm, climb to 10 m, hover 30 s, land	passed (also from a clean shell); 1,074 MB RAM (107 MB per drone), 76.5 % CPU during hover	reports/PHASE_1.md
10 drones fly a 1 km V mission and land (6 runs)	formation RMS max 1.24-1.66 m (limit 2), mean 0.54-0.62 m; closest pair 7.80-8.65 m (limit 5); leader lands 1.1-1.7 m from the goal	reports/PHASE_3.md, reports/phase3_mission.png

Agent logic (fast simulator, real agent code)

What	Result	Evidence
Unit tests	130 pass, 4 skipped without ROS / PyTorch (with PyTorch: 133 pass, 2 skipped)	python3 -m pytest
1,000 random runs with crashes, radio loss, delays and splits	exactly one leader after convergence in 1,000 of 1,000; median convergence 0.145 s after the last fault (p95 2.12 s, max 2.72 s)	reports/PHASE_2.md
Known weak spot	143 of those 1,000 runs had two drones closer than 5 m; every one contained a random radio split or random link drops (drones that cannot hear each other cannot be pushed apart)	reports/PHASE_2.md

Fault types, 10 drones, 20 runs each (fast simulator). On PX4, Phase 4 was started and stopped (this laptop is too weak for the 55-trial batch); its indicative results agree, except that F5's formation took about 17 s to recover after the heal (reports/PHASE_4.md):

Fault	New leader agreed (median / worst)	Formation back under 2 m	Closest pair	Goal reached
F1 leader killed	1.64 / 1.75 s	4.55 / 4.68 s	9.35 m	20/20
F2 leader's radio lost	1.64 / 1.75 s	4.55 / 4.68 s	8.23 m	20/20
F3 leader low battery (planned handover)	0.17 / 0.20 s	3.14 / 3.23 s	9.35 m	20/20
F4 follower killed	no leader change	1.12 / 1.15 s	7.46 m	20/20
F5 radio split, then healed	0.18 / 0.25 s after the heal	9.93 / 13.75 s	6.23 m	20/20

Swarm size (fast simulator, leader killed at 90 s, 3 runs per size)

Drones	New leader	Formation back	Closest pair	All landed	Simulator speed	Heartbeat
2	1.60 s	-	10.0 m	3/3	1,506x real time	44 B
10	1.70 s	4.50 s	9.49 m	3/3	162x	45 B
50	1.75 s	4.95 s	9.49 m	3/3	11x	50 B
100	1.75 s	8.75 s	9.49 m	3/3	2.9x	56 B

With 10 % message loss, 150 ms delay and wind-like drift, 100 drones: new leader in 2.75 s median (worst 2.95 s), never two leaders at once, closest pair 8.14 m, all landed. Evidence: reports/SCALING.md , reports/logs/scaling/ .

Obstacle avoidance: learned policy vs classical

Fair comparison: 90 courses never used in training or model selection (30 per density), 10 drones, every method on the same courses with the same wind-like drift. Success = no drone hits anything and the whole V re-forms at the target. Classical = potential field + stopping-distance brake, tuned on training courses; RL = PPO policy after 6 million steps (38 min on the laptop CPU).

Method	Few obstacles	Medium	Dense	Crashes per mission (medium)	Drones left behind (medium)
No avoidance	3/30	0/30	0/30	6.30	0.00
Brake only	8/30	3/30	0/30	2.63	0.80
Classical	15/30	7/30	3/30	0.57	1.20
RL	13/30	7/30	1/30	1.80	0.00
RL + brake	26/30	18/30	5/30	0.47	0.23

Paired on the same courses, RL + brake against classical: few obstacles 13 courses only RL + brake completed vs 2 only classical (exact McNemar $p = 0.007$); medium 11 vs 0 ($p = 0.001$); dense 5 vs 3 ($p = 0.73$, no clear difference). The classical controller also let drones come as close as 0.88-2.04 m to each other; RL + brake stayed at 3.0 m or more. Evidence: reports/logs/rl/eval/ (summary.md , summary.json , episodes.jsonl , comparison.png), tuning reports/logs/rl/apf_tuning.jsonl , training reports/logs/rl/run1/ , reports/rl_training.png .

What it means: the policy alone is about as good as the classical controller but fails differently (it keeps every drone with the formation, but crashes more); with the same brake on top it is clearly better at low and medium density. Dense clutter is unsolved by every method. Training was still improving when it stopped, which is why longer training (Kaggle) is the next step.

On the real map, with the full agent code

F-9 Park to Faisal Mosque, Islamabad (3.2 km; 932 buildings and 26 woods from OpenStreetMap, low flight height so all of them are obstacles), 10 drones, 5 runs with drift; in the second set the leader is killed halfway. Drones that hit something over 5 runs / runs with no hit:

Method	Leader alive	Leader killed halfway
No avoidance	45 / 0 of 5	40 / 0 of 5
Classical (tuned)	10 / 1 of 5	8 / 0 of 5
Classical (default settings)	3 / 2 of 5, but many drones left far behind (formation error ~240 m)	3 / 3 of 5, same caveat
RL	20 / 0 of 5	18 / 0 of 5
RL + brake	0 / 5 of 5	0 / 5 of 5

Every mission completed; after the leader was killed a new leader was agreed in 1.55-1.65 s. These numbers are from the re-run of 27 September 2026 with the current code, in which the agent runs its avoider 10 times a second - the rate at which the policy was trained and the classical controller was tuned. Evidence: `reports/logs/rl/route_eval_10hz/`. The first run (26 September, avoider every 0.05 s) gave the same result for RL + brake (0 hits in all 10 runs) and for RL (20 and 18), but fewer hits for the classical controller (tuned: 4 / 3 of 5 and 7 / 1 of 5; default: 2 / 3 of 5 and 0 / 5 of 5); at twice the update rate the classical controller does better, still not as well as RL + brake. Evidence of that run: `reports/logs/rl/route_eval/`.

Long routes

Route	Setup	Result
Islamabad F-9 Park -> Rawalpindi Saddar (33.5973, 73.0479), 12.01 km	normal height (30 m; the 266 OSM buildings and woods are all below 25 m), RL + brake, 2 charging stops	10/10 drones landed at the target after 2,795 simulated seconds, no hits, both stops used; formation error while cruising max 1.42 m (mean 0.93 m); closest pair 8.53 m
same route	low height (16 m; all 253 buildings and 13 woods are obstacles), RL + brake, 3 stops	leader route 13.51 km around them; mission completed after 3,131 s; drones 7, 9 and 10 hit buildings (7 of 10 landed); closest pair 5.67 m

Both 12 km runs were repeated on 27 September 2026 with the current code (`scripts/run_route.py`) and gave the same outcome as the first runs of 26 September (2,794.6 s and 10 landed; 3 hits by the same three drones). At 16 m the three drones that crashed had first got stuck about 1.5 m from a wall, crawling at 0.5-3 m/s and falling 100-1,250 m behind the formation, before they hit that building; this is why the formation error while cruising was above 50 m for 446 of 2,704 s (mean 47 m, max 534 m) although the rest of the V held. Evidence: `reports/logs/long_route/rawalpindi_12km_auto.json`, `rawalpindi_12km_low.json`. | Islamabad -> Lahore, 272.55 km | normal height, 38 buildings of 25 m or more from OSM, RL + brake, 65 charging stops | stopped on purpose after 76.0 km: 18 of 65 stops used, 10/10 drones flying, no hits (below) |

Islamabad -> Lahore (stopped early on purpose)

Set-up: 10 drones from F-9 Park, Islamabad (33.7036, 73.0231) to Lahore (31.5204, 74.3587), seed 1, 5 m/s, battery endurance 25 min, flight height "auto" (normal, 30 m on a route this long), avoidance RL + brake. Map data: OpenStreetMap buildings of 25 m or more along the route, downloaded in 14 boxes of 20 km with the light "tall buildings only" query: 38 buildings are in the way, 680 lower ones are flown over. Leader route 272.58 km around them; 65 charging stops about every 4.1 km (half a battery per leg, 3-minute battery swap).

When	Simulated time	Stops used	Flown	Left	Drones flying	Obstacle hits
progress check	7,169 s	7 of 65	-	-	10/10	0
progress check	14,156 s	14 of 65	59.4 km	213.1 km	10/10	0
stopped	18,088 s (5 h 01 min)	18 of 65	76.0 km	196.6 km	10/10	0

Drone 1 was still the leader when it was stopped. The run went at about 16-38x real time (speed "Max"); the remaining ~48,000 simulated seconds would have taken about 20-25 more minutes at the 38x it had reached.

Why it was stopped: it was a scale demonstration, not a new test - the charging-stop mechanism had already completed end to end on the 12 km route above and in `tests/test_long_route.py` - and the owner asked to stop it on 26 September 2026 so the laptop was free for the remaining work. What it shows: the long-route machinery (strip download, chunked route planning, 65 charging stops, landing and take-off at every stop, flying 76 km without a hit or a lost drone) works at city-to-city scale. Not measured: formation error and closest pair over the whole run (the app only shows them live), and the remaining 196.6 km. Evidence: `reports/logs/long_route/Lahore_stopped_2026-09-26.json` (values read from the running app; the app keeps no log files). To finish it: `docs/RUNBOOK.md`, section 4.

Ground-control app

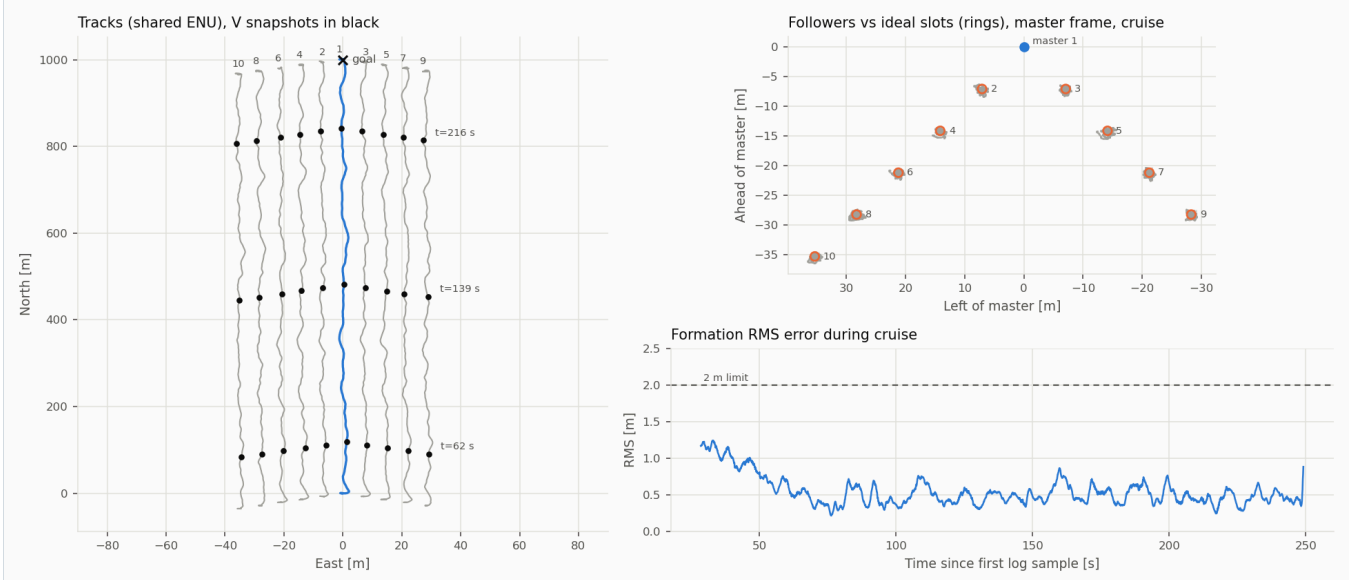
Live 2-D map (Mapbox or Esri satellite, OSM streets), 3-D view (Cesium terrain, OpenStreetMap 3-D buildings or Google photorealistic tiles, 3-D drone models with smooth motion, V-formation lines, trails, chase / orbit / top / free cameras, fault buttons and event banners), real GPS home and target, 1-100 drones, 8 fault types per drone, radio split, speed 1x-Max, obstacle maps and charging stops on the map, results and documents inside the app. When a new leader takes over, the event log says how long after the fault it happened (for example "Drone 2 became master (term 2, election), 1.5 s after the fault on drone 1"; checked by `tests/test_gcs_backend.py`).

Key figures



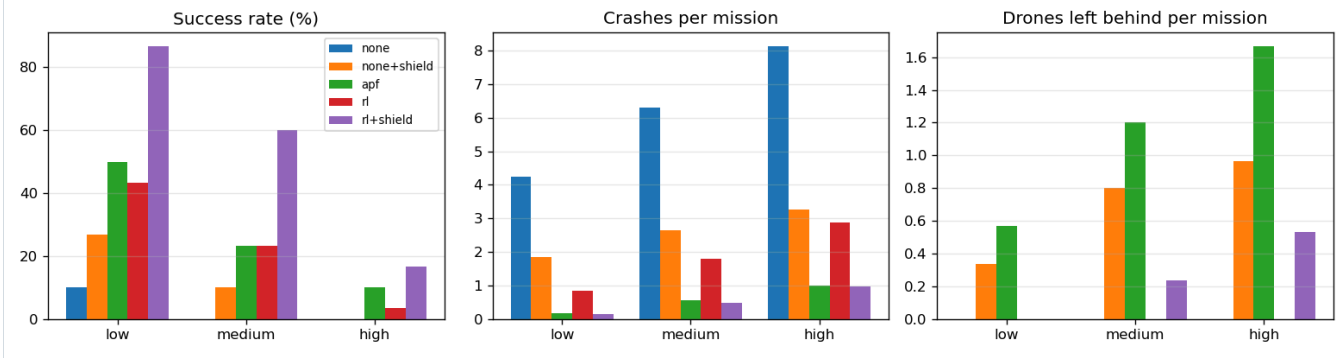
The 3D view of the ground-control app: ten drones in V formation over F-9 Park, Islamabad (Chase camera, 27 September 2026). Leader 1 has the blue ring; V lines, height lines and short trails; red = OpenStreetMap buildings that are obstacles at 16 m, green = woods, yellow = the leader's planned route to Faisal Mosque.

10 PX4 drones, 1 km V mission (cross-check run 2)

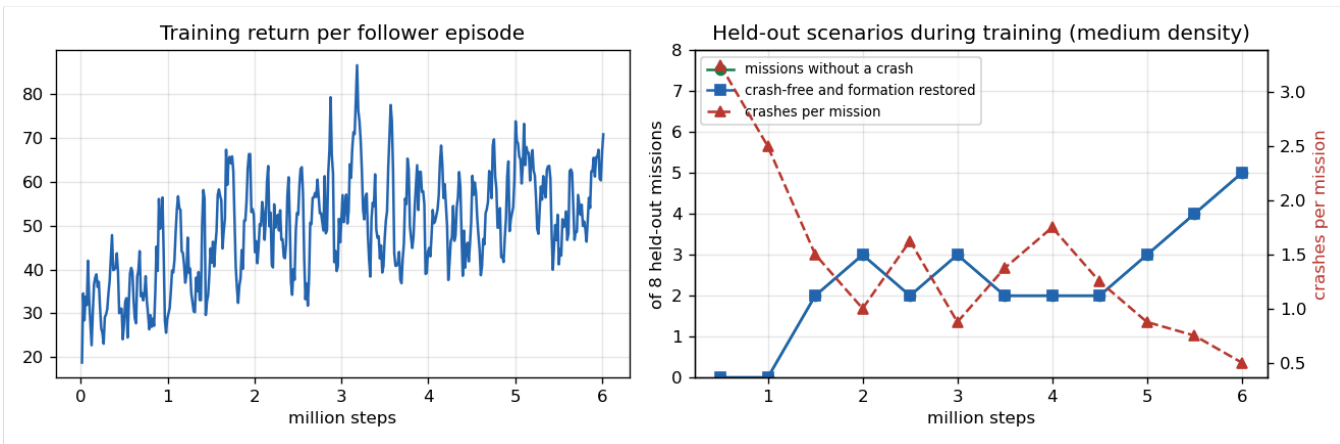


Ten PX4 drones fly a 1 km V mission: tracks with V snapshots, every follower against its ideal slot, formation error over time (Phase 3, cross-check run 2).

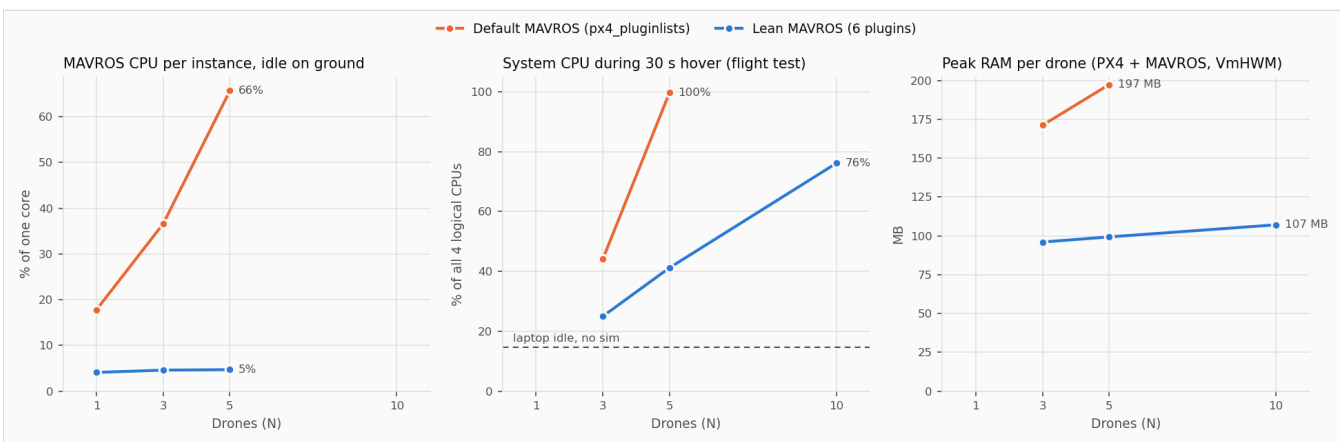
Obstacle avoidance in formation, 10 drones, 30 held-out routes per level



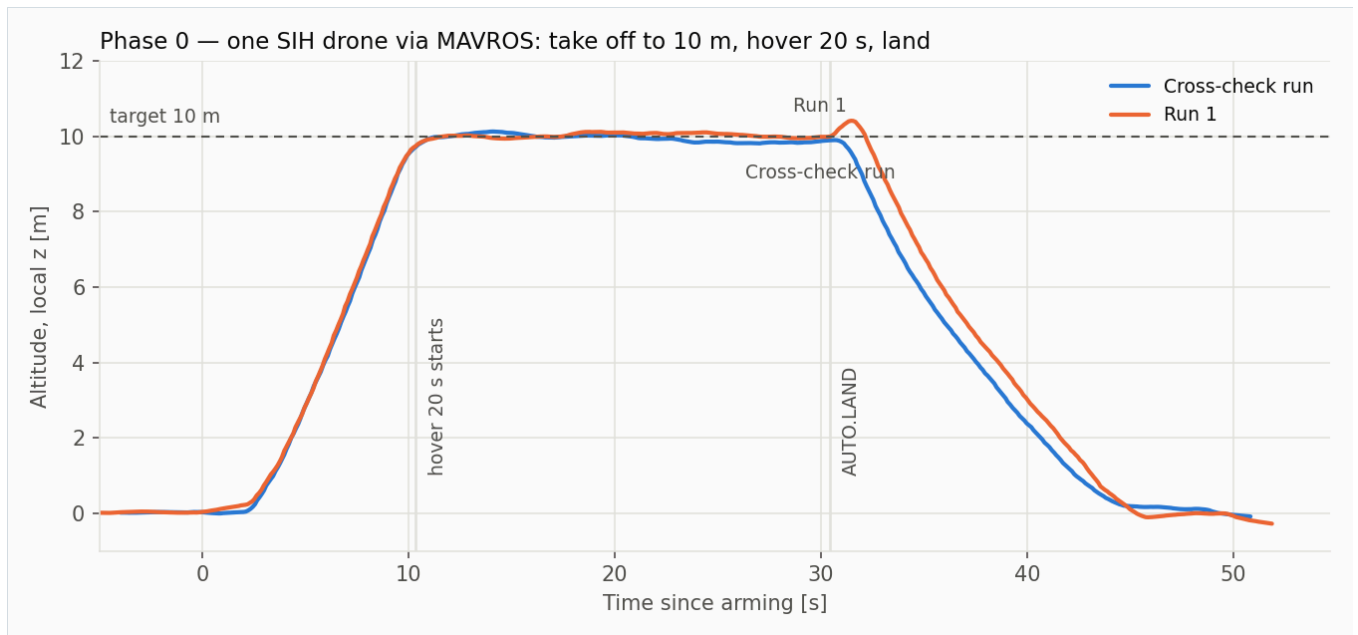
Obstacle avoidance on 90 unseen courses: success rate, crashes and drones left behind for five methods at three obstacle densities.



RL training on the laptop: return per episode and held-out scores; still improving at 6 million steps.



PX4 + MAVROS resources for 3, 5 and 10 drones on this laptop (Phase 1).



The first PX4 flight: arm, climb to 10 m, hover 20 s, land (Phase 0).

How to install and run everything

DOCS/RUNBOOK.MD

How to install and run everything

All commands run from the repository root. The path of this repository contains a space (drone swarm): quote it in shells, and note that PX4 itself cannot run from such a path, so its working folders go to `/tmp/swarm_sim/` (`config/swarm.yaml -> sim.work_dir`).

1. Install from scratch (Ubuntu 24.04)

Steps that need `sudo` must be run by the owner of the machine.

- ROS 2 Jazzy (desktop or base) from packages.ros.org, then `sudo apt install ros-jazzy-mavros` (installed here: `ros-jazzy-mavros` and `ros-jazzy-mavros-msgs` 2.15.1).
- GeographicLib datasets for MAVROS (egm96-5 geoid and the magnetic model): `sudo /opt/ros/jazzy/lib/mavros/install_geographiclib_datasets.sh`. When its SourceForge download fails (it did here), download the files by hand and copy them to `/usr/share/GeographicLib/` (record: `reports/logs/phase_0/sudo_installs.log`, `geographiclib_datasets.txt`).
- PX4: `git clone https://github.com/PX4/PX4-Autopilot.git ~/PX4-Autopilot --recursive`, `cd ~/PX4-Autopilot && git checkout v1.18.0-rc1 && git submodule update --init --recursive`, `bash ./Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools` (`sudo`), then from this repository `scripts/build_px4.sh` (builds `px4_sitl_sih` with 2 jobs; v1.17 has no `px4_sitl_sih`).
- Python packages from Ubuntu (present here already): `numpy`, `scipy`, `matplotlib`, `pyyaml`, `psutil`, `pytest` (`sudo apt install python3-numpy python3-scipy python3-matplotlib python3-yaml python3-psutil python3-pytest python3-pytest-cov`).
- Only for RL training on the laptop: `python3 -m venv .venv --system-site-packages`, `.venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cpu`, `.venv/bin/pip install gymnasium stable-baselines3` (about 0.9 GB).
- Optional map keys: copy `config/map_keys.example.yaml` to `config/map_keys.local.yaml` and paste your Mapbox and Cesium ion tokens (never commit that file).
- Desktop icon: `bash scripts/install_launcher.sh`.

Check the environment: `scripts/env_audit.sh`. Before any PX4 run: `free -m` should show more than 800 MB available; kill leftovers with `pkill -x px4`; `pkill -x mavros_node`.

2. Every day

Task	Command
Open the ground-control app	double-click Swarm Control in the project folder or on the desktop (or Start Swarm Control.sh : right-click → Run as a Program), or <code>python3 scripts/gcs.py</code> and open <code>http://localhost:8080</code>
Stop it (when started by a launcher)	<code>scripts/stop_swarm.sh</code>
Install the launchers (once per machine, and again after moving the folder)	<code>bash scripts/install_launcher.sh</code> : desktop and menu icon, and the Swarm Control program in the folder (needs <code>gcc</code> ; the program is not in git, so a fresh copy of the repository needs this once)
Run all tests	<code>python3 -m pytest</code> (130 pass, 4 skip without ROS/PyTorch); with PyTorch: <code>PYTHONPATH=src .venv/bin/python -m pytest</code> (133 pass, 2 skip)

In the app: choose home and target (type or pick on the map), number of drones (1-100), obstacles (none or real buildings), avoidance, flight height; Start; change the speed; use the fault buttons (kill leader, kill 2/3/4, per-drone crash / motor / battery / GPS / radio faults, radio split). Routes up to 400 km; longer than a battery leg they get charging stops automatically.

The **3D view** page follows the same simulation: the drones are 3-D models (leader blue with a ring, followers white, leaving amber, no radio purple, down red), with V-formation lines, trails, and a line down to the ground under each drone. Cameras: **Chase** (behind the formation), **Orbit** (circles it), **Top** (straight down) — in these three, drag to turn and use the wheel to zoom — and **Free** (move the camera yourself). Switches for trails, V lines, labels, obstacles, 3-D buildings, sun shadows and Google photorealistic 3-D (heavy). Fault buttons: kill the leader, kill 2 at random, leader battery low, cut the leader's radio, split the radio net, restore radios. Big events (a drone down, a new leader and how many seconds after the fault) appear as a banner, and the last events are listed at the top right.

3. PX4 simulation (ROS 2 environment through `scripts/ros_env.sh`)

Phase	Command	Output
0: one drone, 10 m hover	<code>scripts/ros_env.sh python3 scripts/phase0_flight_test.py</code>	<code>reports/logs/phase_0/</code>
1: N drones hover	<code>scripts/ros_env.sh python3 scripts/phase1_scale_test.py --n 10</code>	<code>reports/logs/phase_1/n10/</code>
3: formation mission	<code>scripts/ros_env.sh python3 scripts/run_mission.py --n 10 --run-dir reports/logs/phase_3/run1</code>	run folder: <code>states.jsonl</code> , <code>metrics.json</code> , ...
3: acceptance batch (3 runs + 3 clean-shell cross-checks)	<code>bash scripts/phase3_runs.sh</code>	<code>reports/logs/phase_3/</code>
4: fault trials (not completed; needs a stronger computer or N=5, ~7 h)	<code>bash scripts/phase4_all.sh 1, then python3 scripts/phase4_metrics.py --out ... --md ... reports/logs/phase_4/F?t* and python3 scripts/plot_phase4.py</code>	<code>reports/logs/phase_4/</code> ; how-to: <code>reports/PHASE_4.md</code>
Metrics / plots of a run	<code>python3 scripts/metrics.py <run_dir>; python3 scripts/plot_mission.py <run_dir> out.png</code>	
Replay page from a run	<code>python3 scripts/make_replay.py <run_dir> scripts/replay_template.html reports/replay/swarm_replay.html</code>	open in the app: PX4 flights
Logs as CSV	<code>PYTHONPATH=src python3 -c "from swarm_tools.logfmt import jsonl_to_csv; jsonl_to_csv('<run>/states.jsonl', '<run>/states.csv')"</code>	

Phase	Command	Output
Phase 0 / Phase 1 figures	<code>python3 scripts/plot_phase0.py; python3 scripts/plot_phase1.py</code>	<code>reports/phase0_altitude.png</code> , <code>reports/phase1_resources.png</code>
Diagnostics: CPU and memory of idle PX4 + MAVROS for several N (Phase 1)	<code>scripts/ros_env.sh python3 scripts/mavros_idle_scaling.py --ns 1 3 5</code>	console
Diagnostics: where 2 SIH drones spawn without home settings (Phase 1)	<code>scripts/ros_env.sh python3 scripts/spawn_default_check.py</code>	console

Every PX4 drone listens on UDP 14580+n and sends to 14540+n (PX4's own port plan, which covers 10 instances); MAVROS runs in namespace `/uav<id>`. Serial connection URLs are refused by the tools.

4. Fast-simulator experiments (no ROS)

Experiment	Command
1,000 random fault runs (Phase 2)	<code>PYTHONPATH=src python3 scripts/random_trials.py --runs 1000 --workers 3</code>
Fault types F1-F5	<code>PYTHONPATH=src python3 scripts/puresim_faults.py --n 10 --seeds 20</code>
Swarm size 1-100	<code>PYTHONPATH=src python3 scripts/scale_test.py --ns 1 2 3 5 10 20 50 100 --seeds 1 2 3 --jsonl reports/logs/scaling/fastsim_scaling.jsonl</code>
One app mission without the browser, saved as JSON (events, progress, formation error, closest pair)	<code>PYTHONPATH=src python3 scripts/run_route.py --target LAT LON [--altitude auto\ low\ normal] [--avoider rl+shield] --out reports/logs/long_route/NAME.json</code>
The 12 km route (F-9 Park -> Rawalpindi Saddar), ~2 min each	<code>... scripts/run_route.py --target 33.5973 73.0479 --altitude auto --out reports/logs/long_route/rawalpindi_12km_auto.json</code> , and <code>--altitude low</code>
Islamabad -> Lahore to the end (272.55 km, 65 stops), ~30-60 min	<code>PYTHONPATH=src python3 scripts/run_route.py --target 31.5204 74.3587 --out reports/logs/long_route/lahore.json</code>

5. Obstacle avoidance and RL

Step	Command	Time on this laptop
Tune the classical avoider (training scenarios)	<code>PYTHONPATH=src python3 scripts/rl_tune_apf.py</code>	~15 min
Train the policy (CPU, PPO)	<code>PYTHONPATH=src .venv/bin/python scripts/rl_train.py --steps 6000000 --run reports/logs/rl/run1</code>	~40 min
Learning curve	<code>python3 scripts/plot_rl_training.py reports/logs/rl/run1 --out reports/rl_training.png</code>	seconds
Fair comparison, 90 unseen courses	<code>PYTHONPATH=src python3 scripts/rl_eval.py --policy models/avoid_policy.npz --episodes 30 --jobs 3</code>	~9 min
Real map, full agent code	<code>PYTHONPATH=src python3 scripts/rl_eval_route.py --policy models/avoid_policy.npz --seeds 5 --jobs 3</code>	~8 min
Use a policy in the app	copy its <code>best_policy.npz</code> to <code>models/avoid_policy.npz</code>	
Long training on a GPU	step by step in <code>docs/KAGGLE.md</code>	~11 h per seed on Kaggle
Learning curve of a Kaggle run	<code>python3 scripts/plot_rl_training.py reports/logs/rl/kaggle_seed1 --out reports/rl_training_kaggle_seed1.png</code>	seconds

6. Documents

Document	Command
Project brief page	<code>PYTHONPATH=src python3 scripts/make_brief.py -> reports/brief/swarm_failover.html</code>
Handover PDF	<code>PYTHONPATH=src python3 scripts/make_handover_pdf.py -> docs/HANDOVER.pdf</code> (needs Google Chrome)
Online copy of the project brief	a copy of <code>reports/brief/swarm_failover.html</code> was published once (26 September 2026) as a private online page in the owner's account; it is not updated automatically - republish it after regenerating the brief
Kaggle code bundle	<code>python3 scripts/make_kaggle_bundle.py -> kaggle/swarm-rl-code.zip</code> (never contains <code>config/map_keys.local.yaml</code>)
Kaggle guide alone as a PDF	<code>PYTHONPATH=src python3 scripts/make_handover_pdf.py --only kaggle --out docs/KAGGLE_GUIDE.pdf</code>
3-D drone model of the 3D view	<code>python3 scripts/make_drone_model.py -> src/swarm_tools/gcs/static/drone.glb</code>

7. When something goes wrong

Symptom	Cause and fix
PX4 exits at start with a path error	the repository path has a space; PX4 must run from <code>/tmp/swarm_sim/...</code> (done by <code>sim_launch.py</code>)
A PX4 build uses all cores and the laptop freezes	always build with <code>scripts/build_px4.sh</code> (2 jobs)
Drones do not arm / MAVROS not connected	leftovers from an earlier run: <code>pkill -x px4; pkill -x mavros_node</code> , check <code>free -m</code>
"Downloading buildings ... map server busy"	the public OpenStreetMap (Overpass) servers are overloaded; wait or use the normal flight height (light query); downloads are cached in <code>data/osm/</code>
3D view says "Cesium key needed"	paste the Cesium ion token into <code>config/map_keys.local.yaml</code> and reload
3D view dark or blurred for a few seconds after opening	the terrain tiles are still loading; wait, the camera glides in on its own

Symptom	Cause and fix
3D view slow, laptop fan loud	switch off Photorealistic and Sun shadows; close the 3D tab during long simulations
Double-clicking <code>Start Swarm Control.sh</code> opens a text editor	right-click → Run as a Program, or use the Swarm Control program next to it
The desktop or menu icon does nothing after the folder was moved	the icon stores the folder's path: run <code>bash scripts/install_launcher.sh</code> again (the Swarm Control program inside the folder finds its own place and keeps working)
Satellite map is Esri, not Mapbox	no Mapbox token in <code>config/map_keys.local.yaml</code> , or the server was still starting (reload)
RL options greyed out in the app	<code>models/avoid_policy.npz</code> is missing
The app is slow with obstacles	expected: about 15x real time with 10 drones and the RL avoider; open sky is ~150x

Decisions and why

DOCS/DECISIONS.MD

Decisions and why

The decisions that shape the system, what was chosen, and the reason. Where a decision came from a measured problem, the evidence is named.

#	Decision	Why
1	No central controller. Every drone runs the same agent and decides its role from heartbeats.	The goal is surviving the loss of any drone, including the leader; the same code must later run on each real drone (docs/SPECIFICATION.md section 6).
2	MAVROS (MAVLink) per drone, not uXRCE-DDS.	Real drones will talk over low-bandwidth MAVLink telemetry radios; switching a drone from simulated to real should only change its connection URL.
3	PX4 SIH, headless, v1.18.0-rc1.	Runs 10 drones on an 8 GB laptop without Gazebo; px4_sitl_sih exists from v1.18 (not in v1.17).
4	Lean MAVROS plugin list + per-namespace remaps of /tf, /tf_static, /parameter_events.	The default plugin set shared /parameter_events across all drones: 5 drones took 99.7 % CPU; lean plugins cut MAVROS CPU per instance from 65.7 % to 4.7 % (reports/PHASE_1.md).
5	One shared ENU frame from each drone's GPS, never MAVROS local frames.	Each MAVROS local frame starts where that drone powered on; comparing them would silently corrupt formation and separation.
6	Lowest alive eligible ID leads; term counter; MASTER_OK blocks claims; no pre-emption; first election waits for the fleet.	Deterministic and testable: exactly one leader after any split heals (1,000/1,000 runs). Waiting for the fleet at startup fixed drone 3 sometimes leading. Learned methods are not used here on purpose (decision 17).
7	Heartbeat as a compact binary message (43-byte header + member mask).	Fits telemetry radios: 18 kbit/s for 10 drones at 5 Hz. The mask was 64-bit (63 drones max) and became variable-length to allow up to 250 drones (PX4's system-ID limit).
8	Parity arms (even left, odd right) instead of alternating slots.	With alternating slots a single loss swapped every later drone across the formation (moves up to 64 m); parity arms only slide the drones behind the loss (reports/PHASE_2.md).
9	Transit layer (6 m below) for long moves, orphan layer (8 m above) for drones the leader cannot hear.	Crossing paths after a radio split healed caused close passes; vertical separation removes them.
10	Leader flies a fixed-heading leg with cross-track correction and a speed ramp.	Re-aiming at the goal made the leader wander and the V rotate; the ramp removed a formation-error spike at the start of cruise (reports/PHASE_3.md).
11	Emergency landing drops below the formation while keeping its speed.	Landing straight down let the drones behind come within 4.66 m; keeping speed while descending lets them pass over.
12	Pure-Python fast simulator running the real agent code.	Thousands of runs in minutes (1,000-run test, scaling to 100 drones, the app), while PX4 validates the same code with the real autopilot.
13	Obstacles are 2-D at flight height; flight height decides what counts.	Simple and honest: at 16 m every building and wood is in the way; at 30 m only tall structures (OSM height or storeys) are. Flying over is not modelled.
14	Classical global planning for the leader (A*), local avoidance for followers.	The leader's route can be computed from the map; followers must react locally while keeping formation, which is where learning can help.
15	Every avoider gets the same inputs and the same safety layer; the classical one is tuned first.	A fair comparison: the question is whether learning beats a good classical controller, not a weak one.
16	Held-out evaluation seeds, paired statistics, confidence intervals.	Results must hold on courses the policy never saw, and differences must be distinguishable from luck.
17	RL only where it can plausibly help: local obstacle and collision avoidance. Not for leader election, not for motor control.	Election must be provably correct; motor control is already solved by PX4.
18	Residual policy (a correction on top of the formation law) with a hard brake shield.	Starts from sensible behaviour, trains faster, and the brake keeps a learned mistake from becoming a crash; RL + brake was the best method.
19	Policy weights exported to numpy; no ML library on the drone.	The onboard agent stays light and dependency-free; the exported policy matches the trained one to 3e-8.
20	Long routes: strip download, chunked planning, charging stops at open spots, landing in place on low battery.	A single map query or planning grid over hundreds of kilometres is impossible; a quadcopter needs battery swaps about every 4 km; flying home from 100 km away is not realistic.
21	Positions put back on the ground far from home (tangent-plane correction).	Without it a drone 270 km away is drawn about 240 m off.
22	Map keys in a git-ignored local file, never typed in by the assistant.	Keys are the owner's credentials.
23	Everything tunable in config/swarm.yaml; strict config loading.	One source of truth; a missing or unknown key is an error, not a silent default.
24	Simulation only.	No serial ports, no real flight controller, no firmware flashing until the owner decides (docs/SPECIFICATION.md rule 7).
25	The GPU trainer picks its best policy on separate validation courses, never on the test courses.	Choosing among ~20 checkpoints by their test score would inflate the reported result. The laptop trainer already used separate courses; the GPU kit did not until the final check (27 September 2026).
26	Secrets never leave the laptop: the Kaggle zip skips *.local.* files and checks every packed file for key values.	The first zip had included the map keys; uploading it would have handed them to a third-party service.
27	The 3-D drone model is generated by a script (scripts/make_drone_model.py), not downloaded.	No licence questions, a 54 KB file, easy to change; built chunkier than a real frame so it stays readable when drawn about 60 pixels wide.
28	The avoider runs at 10 Hz in the agent, the rate used in training and tuning.	About 2.7 times faster simulation with obstacles (5.8x -> 15.6x real time on the Lahore route, together with a cheaper policy input), and every avoider runs at the rate it was trained or tuned for. The classical avoider did better at 20 Hz on the real map; both results are reported (docs/RESULTS.md).

#	Decision	Why
29	The Islamabad -> Lahore run was stopped at 76 km on purpose.	It only demonstrated scale (the charging-stop mechanism had already completed end to end on the 12 km route); the owner wanted the laptop free for the handover work. How to finish it: docs/RUNBOOK.md , section 4.
30	Phase 4 stopped on this laptop (27 September 2026).	With 10 PX4 drones the i3 laptop runs at 60-100 % CPU on the charger and saturates on battery; the owner stopped the 7-hour batch. Run it on a machine with 8+ cores, or with 5 drones here (docs/SPECIFICATION.md rule 6); everything is in reports/PHASE_4.md .

Known issues, what is not done, next steps

DOCS/KNOWN_ISSUES.MD

Known issues, what is not done, and next steps

Not done (from the original plan in `docs/SPECIFICATION.md`)

Item	State	What it takes
Phase 4: fault trials on PX4 (leader killed, radio lost, low battery, follower killed, radio split; 10 trials each)	started twice on 27 September 2026, stopped by the owner: this laptop saturates with 10 PX4 drones (indicative results: all takeovers within limits; F5 formation recovery ~17 s against 15 s)	<code>bash scripts/phase4_all.sh 1</code> on a machine with 8+ cores (~7 h), or <code>N=5</code> on this laptop; step by step in <code>reports/PHASE_4.md</code>
Phase 5: radio realism sweep (50/150/300 ms x 0/10/30 % loss)	not started	a sweep driver around <code>scripts/run_mission.py</code> with the link emulator settings; several hours
Phase 6: mixed-reality readiness (1 real + N simulated drones, telemetry-radio stand-in, flight test plan, go/no-go checklist)	not started	profiles in <code>config/profiles/</code> , a UDP proxy that limits bandwidth, <code>FLIGHT_TEST_PLAN.md</code>
Other formations (line, column, echelon, diamond, squads) and their comparison	not started	generalise <code>formation.py</code> beyond the V; compare formation error, failover and obstacle results
Longer RL training, several seeds	kit ready and checked on 27 September 2026, not run	one ~11-hour Kaggle GPU session per seed, step by step in <code>docs/KAGGLE.md</code>
Islamabad -> Lahore run to the end	stopped on purpose at 76 km (18 of 65 stops, 10/10 drones, 0 hits)	<code>PYTHONPATH=src python3 scripts/run_route.py --target 31.5204 74.3587 --out reports/logs/long_route/lahore.json</code> (no browser, about 30-60 min), or in the app

Known limitations

- **No real drone has flown this code.** The Pixhawk 6C cannot run it alone; each drone needs a companion computer (for example a Raspberry Pi 4/5) connected to the Pixhawk by serial cable, and a radio link between drones for the heartbeats (a bridge node replacing the link emulator).
- **Separation when drones cannot hear each other.** In 143 of 1,000 random fast-simulator runs, two drones came closer than 5 m; every such run had a random radio split or random link drops. The rules cannot push apart drones that do not hear each other. Needs onboard proximity sensing or sticky formation references before real flights.
- **Dense clutter is unsolved.** In the densest obstacle courses the best method (RL + brake) completes 5 of 30; followers still hit buildings in dense city blocks at 16 m.
- **Simplified physics.** PX4 SIH uses a simple aircraft model; the fast simulators use point masses with a first-order velocity response and a random drift instead of real wind.
- **Perfect sensing assumed.** Positions, velocities and range readings are exact in simulation. Real GPS drifts by metres (use RTK GPS or wider spacing); range readings would come from a real sensor (lidar or depth camera) or from a map that may be out of date.
- **2-D obstacles.** Obstacles are footprints at the flight height; flying over is not modelled, OSM heights are often missing (defaults: houses 9 m, woods 15 m).
- **Map data availability.** The public Overpass servers are often busy (HTTP 429/504); downloads retry and are cached, but a first long-route download can take tens of minutes.
- **Radio load.** Heartbeats go from every drone to every drone: 224 kbit/s for 100 drones, far more than one telemetry radio channel (SiK default 64 kbit/s). Large swarms need a mesh network, lower rates, or squads.
- **One V for large swarms.** With 100 drones each arm is about 490 m long; big swarms should fly as squads of about ten.
- **PX4 on this laptop is limited to 10 drones** (CPU, and PX4's default port plan).
- **Phase 3 harness:** two runs ended with the harness's watcher thread stalled (flights fine); the fix is in the code but has not been re-run on PX4.
- **Followers can get stuck against buildings when flying low.** On the 12 km route at 16 m, three followers got stuck about 1.5 m from a wall, crawled, fell 100-1,250 m behind the formation and finally hit the building (7 of 10 landed). At the normal 30 m the same route is clean (formation error max 1.42 m, no hits). A fix needs a way out of dead ends (for example re-planning a short path for a stuck follower) - the learned avoider only sees 25 m around it. Evidence: `reports/logs/long_route/`.
- **The classical avoider depends on its update rate.** On the real map it hit 4 and 7 buildings when run every 0.05 s but 10 and 8 at 10 Hz, its tuned rate; RL + brake had 0 in both (`docs/RESULTS.md`).
- **F5 on PX4: the formation re-forms slowly after a radio split heals.** The group with the newer leader wins (the term rule), so the whole V re-forms around it; the transit layer and PX4's vertical speeds make that take about 17 s against a 15 s target (`reports/PHASE_4.md`, three options).
- **This laptop's power.** On battery the power-saver profile throttles the CPU and 10 PX4 drones saturate it; the battery also ran out twice on 27 September 2026. Run long PX4 jobs only on the charger (`scripts/phase4_runs.sh` waits for it).
- **3-D view heights.** The simulators have flat ground; the 3-D view draws every drone at its simulated height above the real terrain under it (Cesium World Terrain), so on hills it shows height above ground, not a real climb. For the first seconds after the page opens the terrain is still coarse and the picture can be dark or blurred until the tiles arrive.
- **3-D view load.** Google photorealistic tiles and sun shadows are heavy for this laptop's integrated GPU; keep them off while a long simulation runs. The page draws at most 40 frames per second; above 20 drones only the leader gets a trail and a label.

Road to real drones (in order)

1. Phase 4 and Phase 5 on PX4 (fault trials, bad radio).
2. Fix the separation weak spot for drones that cannot hear each other.
3. Measure the real drone: weight, battery capacity, motor and propeller data, PX4 flight logs; put the numbers into the simulators (a digital twin), re-run the tests.
4. Companion computer per drone (Ubuntu 24.04, ROS 2 Jazzy, MAVROS, this agent), wired to the Pixhawk 6C; a heartbeat radio bridge (WiFi mesh or telemetry radios).

5. Bench test with propellers off; then one real drone with nine simulated ones (Phase 6), RC override and a kill switch on every real drone, PX4 geofence and failsafes configured.
6. Three real drones in an open field with safety pilots; only then more.
7. Legal permission for flights (local civil aviation rules) before any outdoor test.

Also before real flights: PX4 is pinned to a release candidate (v1.18.0-rc1, the first version with the `px4_sitl_sih` target); move to a stable PX4 release for the real drones and run Phases 0-4 again.

Ideas for later

- RL where mistakes are cheap: area re-division when drones fail (search and rescue), charging and patrol schedules, battery-aware routing - planning on the ground rather than flight control.
- Train on real OpenStreetMap neighbourhoods instead of random courses; train with the brake in the loop; curriculum from sparse to dense clutter.
- Terrain heights (for example Mapbox Terrain-DEM or Copernicus) so the swarm keeps its height above hills such as the Margalla range.

Housekeeping (not done)

- **No backup.** The repository has no Git remote: the only copy of the code, the logs and the history is on this laptop, which lost power twice on 27 September 2026. Push it to a private remote (for example a private GitHub repository) or copy the folder to an external drive. Git-ignored files are not in the history and need their own copy if wanted: `config/map_keys.local.yaml` (keys), `.venv/`, `kaggle/swarm-rl-code.zip` (rebuilt by a script) and the `Swarm Control` launcher (rebuilt by `scripts/install_launcher.sh`).
- **No licence file.** Choose a licence before sharing the code outside the team. Third-party data keep their own terms: OpenStreetMap data (ODbL, attribution), Mapbox and Cesium ion / Google tiles (their terms of service, shown as credits in the app).
- **The online project brief is out of date** (published 26 September 2026; see `docs/RUNBOOK.md`, section 6).

Found and fixed during the final check (27 September 2026)

- **Kaggle bundle packed the private map keys.** `scripts/make_kaggle_bundle.py` zipped the whole `config/` folder, including `config/map_keys.local.yaml`. The zip had not been uploaded. The script now skips `*.local.*` files and refuses to build if any packed file contains a key value; `tests/test_tools_scripts.py` checks both.
- **GPU training would have chosen its best policy on the test courses.** `build_pools.py` gave the trainer the same 90 courses the final comparison reports on. Now the trainer scores separate validation courses (`val.npz`, seeds 1,000,000-1,000,029); the test courses (1,000,100-1,000,129) are used only for the final report (checked by `tests/test_tools_scripts.py`).
- **3-D view:** drones were drawn before the terrain height was known, so their trails started hundreds of metres below the ground and the camera could end up under the terrain. Drones now appear once the ground height is known, trails are stored as height above ground, and the camera lifts itself above hills.
- **Learning-curve plot** could not read the GPU trainer's logs; `scripts/plot_rl_training.py` now reads both formats.
- **Stopping the app did not stop it.** `scripts/start_swarm.sh` saved the process ID of a helper shell instead of the app, so `scripts/stop_swarm.sh` said "stopped" while the app kept running. The start script now saves the app's own ID, and the stop script checks that the process really is the app (and falls back to looking for it) before and after stopping it.
- **In-app Docs page showed no documents.** `static/md.js` had a JavaScript syntax error ("use strict" inside a function with a default parameter). Fixed; the viewer also shows pictures and quote boxes now.

Long RL training on Kaggle

DOCS/KAGGLE.MD

Long RL training on Kaggle — the complete guide

Status (27 September 2026): everything is ready, nothing has been run on Kaggle yet. The kit has only been tested on this laptop with tiny settings (a few hundred training steps on the CPU, 16 courses), so the first real Kaggle run may still show a problem that only appears there (paths, GPU memory, time). This file explains, step by step: which files to take, how to start the training on Kaggle, what gets trained, what problem it solves, how to change things, where to paste every file when training is finished, and what you get at the end. Section 12 has the whole procedure on one page.

In short. One command on the laptop builds a zip file. Upload it to Kaggle as a dataset, import our notebook, switch the GPU on and press "Save & Run All". About 11 hours later Kaggle gives you `results.zip`, which holds the newly trained policy (`best_policy.npz`) and its test results. If the new results are better than the old ones, put `best_policy.npz` in place of `models/avoid_policy.npz`: the app and the drones then use the new policy on their own. Details in section 12 and below.

1. What gets trained (in simple words)

Only one thing: the **obstacle-avoidance brain of the followers** — a small neural network.

- **Inputs (47 numbers, 10 times a second):** where the formation wants the drone to go (desired velocity), the drone's own velocity, how far it is from its place in the V, 24 distance readings around the drone (like a 2-D lidar, up to 25 m), the closest obstacle point, and the three nearest drones. Everything is turned into the formation's own frame (forward/left), so the same network works whichever way the route points (`src/swarm_agent/avoidance.py`).
- **Output (2 numbers):** a sideways/forward velocity correction (up to 8 m/s per axis) added to the formation command. The hard safety layer (drone-to-drone repulsion, geofence) and — in the "RL + brake" option — a stopping-distance brake stay on top of it.
- **Network:** 47 inputs → 128 → 128 (tanh) → 2 outputs. After training it is saved as plain numpy weights (`.npz`, about 180 KB), so the drone needs no machine-learning library.
- **How it learns:** PPO (a standard reinforcement-learning method). One shared network drives all nine followers. On the GPU, 1024 missions run at the same time (`src/swarm_tools/torch_sim.py`, checked step by step against the laptop simulator by `tests/test_torch_sim.py`).
- **What it is rewarded for (per follower, every 0.1 s):** staying close to its slot in the V, getting closer to it, small corrections; it is penalised for coming within 4 m of an obstacle, for coming closer than 5 m to another drone, and heavily (−10) for a crash. The exact formula is in `src/swarm_tools/obstacle_sim.py` (search for `# rewards for the followers`).
- **Training courses:** random routes of 300–600 m through random buildings and tree clusters (0–2 buildings and 0–1 tree clusters per hectare), 8,000 different courses.

What is **not** trained: the leader election, the formation law, the leader's route planner (A*), the autopilot. Those are rules and stay exactly as they are (see `docs/DECISIONS.md`, 14 and 17).

2. What problem it solves, and what it will not

The laptop training (6 million steps, 38 minutes on the CPU) produced the current policy. On 90 test courses it never saw (`docs/RESULTS.md`):

Method	Few obstacles	Medium	Dense
Classical (tuned)	15/30	7/30	3/30
RL + brake (laptop policy)	26/30	18/30	5/30

Two open problems:

1. **The training was still improving when it stopped** (`reports/rl_training.png`). Dense clutter is unsolved (5 of 30). Longer training — about 10 hours on a GPU per session instead of 38 minutes on the CPU — should reduce crashes, most of all at medium and high density. How much is unknown until it is measured.
2. **One training run proves little.** Training again with seeds 2 and 3 shows whether the result repeats; that is needed before anyone presents the result as reliable.

It will **not** fix: the simplified physics, the 2-D obstacles (no flying over), the perfect sensing assumed in simulation, or the radio-split separation weak spot (`docs/KNOWN_ISSUES.md`).

3. What you need

- A Kaggle account (free). Kaggle may ask you to verify a phone number before GPUs can be switched on.
- GPU time: every run uses one session of up to 12 hours (Kaggle's limit for GPU notebooks, from kaggle.com/docs/notebooks). Your weekly GPU quota is shown in your Kaggle account settings; plan one session per seed, so three sessions for seeds 1–3.
- On the laptop: this repository. Nothing has to be installed on Kaggle: PyTorch is already in Kaggle's image and the notebook installs nothing, so it runs with Internet off.

4. Which files to take

Only **two files** go to Kaggle:

File on the laptop	What it is	How it gets to Kaggle
kaggle/swarm-rl-code.zip	all the code (src/, scripts/, config/, cached map data data/osm/, pyproject.toml), about 0.5 MB	uploaded as a private Dataset named <code>swarm-rl-code</code>
kaggle/train_swarm_rl.ipynb	the notebook that runs everything	imported as a notebook

Make the zip fresh every time (it is not in git), from the repository root:

```
python3 scripts/make_kaggle_bundle.py
```

It prints wrote `../kaggle/swarm-rl-code.zip` (N files, 0.5 MB; no `*.local.*` files, no map keys). Your map keys (`config/map_keys.local.yaml`) are never packed: the script skips `*.local.*` files and refuses to build if any packed file contains one of your key values. (Fixed on 27 September 2026 — an earlier zip had included the keys file; that zip was never uploaded.)

5. Starting the training on kaggle.com, click by click

- Upload the code.** kaggle.com → **Create** → **New Dataset** → drag `kaggle/swarm-rl-code.zip` in → title `swarm-rl-code` → keep it **Private** → **Create**. (Kaggle may unpack the zip; the notebook handles both cases.)
- Import the notebook.** **Create** → **New Notebook** → menu **File** → **Import Notebook** → choose `kaggle/train_swarm_rl.ipynb`.
- Attach the code.** In the notebook's right-hand panel: **Input** → **Add Input** → search for your `swarm-rl-code` dataset → **Add**.
- Switch the GPU on.** Right-hand panel → **Session options** → **Accelerator** → **GPU P100** (or **GPU T4 x2**; the trainer uses one GPU). Internet can stay off.
- Choose the seed.** In the first code cell: `SEED = 1` (later runs: 2, 3). Leave `HOURS = 10.5`.
- Run it as a background job:** **Save Version** → **Save & Run All (Commit)** → **Save**. This runs on Kaggle's machines from top to bottom even if you close the browser or switch the laptop off. (Do not just press "Run all" in the editor; an interactive session stops after 20 minutes without activity.)
- Watch it (optional).** Open the notebook's page → the running version → **Logs**. Every few minutes a line like `step 123,456,789 fps 95,000 ret 1.23 success 0.41 crash_free 0.55 ...` appears, and every 30 minutes an `[eval] {...}` line with the scores on the validation courses.

What the notebook does, and how long it takes:

Cell	What happens	Time
1	prints the GPU name (must say <code>CUDA True</code>), unpacks the code	seconds
2	builds the courses: 8,000 training, 90 validation (to pick the best policy), 90 test (for the final report only)	~10 min
3	trains on the GPU for <code>HOURS = 10.5</code> h; checkpoint every 20 min; scores the 90 validation courses every 30 min and keeps the best policy	10.5 h
4	final fair test on the CPU with exactly the laptop's method: 90 test courses (<code>scripts/rl_eval.py</code>) and the real Islamabad map route (<code>scripts/rl_eval_route.py</code>)	~20–30 min
5	packs everything into <code>/kaggle/working/results.zip</code>	seconds

The code cells are numbered from the top (the text cell above them does not count); each one starts with a comment such as `# Cell 3: training on the GPU`.

Total about 11 hours, inside Kaggle's 12-hour limit. The trainer stops itself when `HOURS` are used up and saves everything first, so nothing is lost at the limit.

Fairness rule (important). The best policy is chosen on the 90 *validation* courses (`val.npz`, seeds 1,000,000–1,000,029). The reported results use the 90 *test* courses (`heldout.npz`, seeds 1,000,100–1,000,129) that neither training nor selection ever saw — the same courses as the laptop comparison, so the numbers are directly comparable. Never pass `heldout.npz` to the trainer. (This was also fixed on 27 September 2026; before that the kit would have chosen the policy on the test courses.)

6. When the run is finished: what to download

On the notebook page open the finished version → **Output** tab → download `results.zip` (a few MB). Unzip it, for example to `~/Downloads/results/`. Inside (for seed 1):

In results.zip	What it is
<code>run_seed1/best_policy.npz</code>	the new brain — the policy with the best validation score
<code>run_seed1/policy.npz</code>	the policy at the very end of training (usually not the one to use)
<code>run_seed1/progress.csv</code>	training log: steps, speed, return, success, crashes, losses (one row per update)
<code>run_seed1/eval.jsonl</code>	validation scores every 30 minutes, per density level
<code>run_seed1/ckpt.pt</code>	full checkpoint (network, optimiser, step count) to continue training later
<code>run_seed1/status.json</code> , <code>session_*.json</code>	how far it got, and the exact settings and GPU of the session
<code>eval_seed1/summary.md</code> , <code>summary.json</code> , <code>episodes.jsonl</code> , <code>comparison.png</code>	the fair test : all five methods on the 90 test courses, same table as the laptop's
<code>route_eval_seed1/summary.md</code> , <code>episodes.jsonl</code>	the real-map test: F-9 Park → Faisal Mosque, 932 buildings, 5 runs, with and without the leader killed

7. Where to paste each file (on the laptop)

From results.zip	Paste it here in the repository	Why
the whole <code>run_seed1/</code> folder	<code>reports/logs/rl/kaggle_seed1/</code>	the training record (evidence)
the whole <code>eval_seed1/</code> folder	<code>reports/logs/rl/eval_kaggle_seed1/</code>	the fair comparison

From results.zip	Paste it here in the repository	Why
the whole route_eval_seed1/ folder	reports/logs/rl/route_eval_kaggle_seed1/	the real-map result
run_seed1/best_policy.npz	models/avoid_policy.npz — only if it is better (section 8); first rename the old one to models/avoid_policy_run1.npz	the app, the simulators and the agent all read this file

The same as commands, from the repository root:

```
mkdir -p reports/logs/rl/kaggle_seed1
cp -r ~/Downloads/results/run_seed1/. reports/logs/rl/kaggle_seed1/
cp -r ~/Downloads/results/eval_seed1 reports/logs/rl/eval_kaggle_seed1
cp -r ~/Downloads/results/route_eval_seed1 reports/logs/rl/route_eval_kaggle_seed1
```

Only after the check in section 8 says it is better:

```
git mv models/avoid_policy.npz models/avoid_policy_run1.npz
cp reports/logs/rl/kaggle_seed1/best_policy.npz models/avoid_policy.npz
```

For seeds 2 and 3 use kaggle_seed2, eval_kaggle_seed2, ... in the same way.

8. Is the new policy better? (decide before replacing)

Open reports/logs/rl/eval_kaggle_seed1/summary.md next to the laptop's reports/logs/rl/eval/summary.md and compare the **RL + brake** row (laptop: 26/30, 18/30, 5/30).

Use the new policy if all of these hold:

1. RL + brake completes at least as many test courses at every density, and more in total;
2. its crashes per mission are not higher than the laptop policy's at any density;
3. in route_eval_kaggle_seed1/summary.md, RL + brake still has 0 drones that hit something (laptop: 0 in 5 runs, with and without the leader killed).

If only some hold, keep the laptop policy and write the result down anyway (a negative result is still a result). With several seeds, pick the seed whose policy is best on these rules and report all seeds, not only the best one.

9. After replacing: check on the laptop and update the documents

```
# 1. the same test on the laptop (should give the same numbers as eval_seed1; takes ~9 min)
PYTHONPATH=src python3 scripts/rl_eval.py --policy models/avoid_policy.npz --out reports/logs/rl/eval_kaggle_check
PYTHONPATH=src python3 scripts/rl_eval_route.py --policy models/avoid_policy.npz --out reports/logs/rl/route_eval_kaggle_check
# 2. learning curve of the Kaggle run
python3 scripts/plot_rl_training.py reports/logs/rl/kaggle_seed1 --out reports/rl_training_kaggle_seed1.png
# 3. tests still pass
python3 -m pytest
```

Then write the new numbers into docs/RESULTS.md (the table under "Obstacle avoidance"), rebuild the documents and commit:

```
PYTHONPATH=src python3 scripts/make_brief.py
PYTHONPATH=src python3 scripts/make_handover_pdf.py
git add models reports docs && git commit -m "RL: Kaggle seed 1 policy, results"
```

10. What you get at the end

- A new models/avoid_policy.npz. The ground-control app uses it the next time it starts (avoidance options "RL policy" and "RL policy + brake"), and so do the fast simulator, the evaluation scripts and the drone agent — no code change.
- A fair, reproducible comparison against the classical avoider on the same 90 test courses, plus the real-map result, with all raw logs in reports/logs/rl/.
- Learning curves showing whether more training still helps.
- With seeds 2 and 3: evidence that the result repeats (or does not).
- A checkpoint (ckpt.pt) to continue training later without starting again.

11. How to change things

To change	Where	Notes
seed	notebook cell 1, SEED	1, 2, 3 ...; each seed is a separate run
training time per session	notebook cell 1, HOURS	keep ≤ 10.5 so pools + evaluation fit in 12 h
number of parallel missions	cell 3, '--envs', '1024'	lower to 512 on "CUDA out of memory"
network size	cell 3, add '--hidden', '256', '256'	the app reads any number of layers; bigger is slower
learning rate	cell 3, add '--lr', '1e-4'	default 3e-4
total step cap (all sessions)	cell 3, add '--steps', '5e9'	default 3e9 agent steps; the time limit usually stops first
how often it checkpoints / scores	'--checkpoint-min' (20), '--eval-min' (30)	minutes

To change	Where	Notes
exploration noise at the start	'--log-std-init' (-1.0)	lower = calmer start
reward	src/swarm_tools/obstacle_sim.py and src/swarm_tools/torch_sim.py	the two must stay identical: change both, then run <code>PYTHONPATH=src .venv/bin/python -m pytest tests/test_torch_sim.py</code> on the laptop
obstacle density of the training courses	SimCfg.building_density, SimCfg.tree_density in obstacle_sim.py	the test levels (LEVELS in rl_eval.py / build_pools.py) stay fixed so results remain comparable
inputs of the network	avoidance.py observation() and the batched version in torch_sim.py	changes OBS_DIM: old policies stop loading (the loader says so); retrain from scratch
number of drones	'--drones' (10)	also pass --drones to the evaluation scripts

Every code change needs a new zip on Kaggle: run `python3 scripts/make_kaggle_bundle.py`, open your `swarm-rl-code` dataset on Kaggle → **New Version** → upload the new zip; in the notebook's **Input** panel make sure the latest dataset version is attached; then **Save & Run All** again.

Continue a seed that stopped at the time limit (the log ends with `[budget] ... saved` and `status.json` says `"done": false`): in a new version of the notebook, **Add Input** → the previous version's output (search your notebook's name), and insert this cell after cell 2:

```
import glob, shutil
prev = glob.glob('/kaggle/input/**/run_seed1', recursive=True)[0]
shutil.copytree(prev, '/kaggle/working/run_seed1', dirs_exist_ok=True)
```

The trainer finds `ckpt.pt` and continues from the saved step, keeping the best score so far.

12. The whole procedure on one page

What gets trained? Only the followers' obstacle-avoidance policy, a small neural network. Every 0.1 s it reads 47 numbers (where the formation wants the drone to go, its own speed, its distance from its place in the V, the distance to walls and trees in 24 directions, the three nearest drones) and returns 2 numbers: how much to change its velocity. The leader election, the formation rules and the autopilot are not trained; they are rules and stay as they are.

Which problem does it solve? The laptop training ran for 38 minutes and was still improving when it stopped. In dense clutter only 5 of 30 missions succeed today. About 10 hours on Kaggle's GPU should reduce crashes; by how much can only be measured. Seeds 2 and 3 show whether the result repeats.

Which files to take? Only two: 1. `kaggle/swarm-rl-code.zip`: first run `python3 scripts/make_kaggle_bundle.py` on the laptop, which builds a fresh zip (your map keys are never included). 2. `kaggle/train_swarm_rl.ipynb`: the notebook.

How to run it on Kaggle: 1. `kaggle.com` → Create → New Dataset → upload the zip → name `swarm-rl-code` → Private → Create. 2. Create → New Notebook → File → Import Notebook → choose `train_swarm_rl.ipynb`. 3. Right-hand panel: Input → Add Input → add your `swarm-rl-code` dataset. 4. Session options → Accelerator → GPU P100 (or T4 x2). Internet can stay off. 5. Keep `SEED = 1` in the first cell (next time 2, then 3). 6. Save Version → **Save & Run All (Commit)** → Save. Kaggle keeps running even if you switch the laptop off. It takes about 11 hours.

When training has finished: 1. On the notebook's page open that version's **Output** tab → download `results.zip` → unzip it into `~/Downloads/results/`. 2. Paste these folders into the project (or copy the commands from section 7): `- run_seed1/ → reports/logs/rl/kaggle_seed1/ - eval_seed1/ → reports/logs/rl/eval_kaggle_seed1/ - route_eval_seed1/ → reports/logs/rl/route_eval_kaggle_seed1/` 3. Compare the "RL + brake" line in `eval_kaggle_seed1/summary.md` with the old one (26/30, 18/30, 5/30). Only if it is equal or better at every density, crashes are not higher, and the Islamabad route still has 0 hits, switch to the new policy: rename `models/avoid_policy.npz` to `avoid_policy_run1.npz` and copy `run_seed1/best_policy.npz` to `models/avoid_policy.npz`. 4. Re-check on the laptop with the commands in section 9, write the new numbers into `docs/RESULTS.md`, rebuild the PDF, commit.

What you get at the end: a new `models/avoid_policy.npz`. The app's "RL policy" and "RL policy + brake" options, the simulators and the drone agent use it without any code change. Plus a fair comparison (the same 90 test courses as on the laptop), the real-map result, and the training curves.

To change something: seed and hours in the notebook's first code cell; everything else in the third cell (table in section 11). If you change code, build a new zip and upload it as a "New Version" of the Kaggle dataset. To change the reward, change `obstacle_sim.py` and `torch_sim.py` in the same way and run `tests/test_torch_sim.py` to check they still match.

13. When something goes wrong

What you see	Cause and fix
cell 1 prints <code>CUDA False</code>	the GPU is not on: Session options → Accelerator → GPU (phone verification may be needed)
cell 1: <code>IndexError: list index out of range</code>	the <code>swarm-rl-code</code> dataset is not attached: Input → Add Input
<code>CUDA out of memory</code>	use <code>'--envs', '512'</code> in cell 3
the version stops at 12 h without <code>results.zip</code>	<code>HOURS</code> too high or the evaluation was slow: set <code>HOURS = 10</code> , and continue the seed (section 11)
the version shows "Error"	open Logs , read the last lines; fix, make a new zip if the code changed, Save & Run All again
<code>policy expects N inputs, this code builds 47</code> when loading a policy	the policy was trained with a different input layout; retrain, or use the matching code version
the laptop check gives different numbers than <code>eval_seed1</code>	should not happen (same scripts, same courses); check that the laptop code is the same version as the zip
the app does not offer the RL options	<code>models/avoid_policy.npz</code> is missing
training <code>success</code> in the log stays at 0 for hours	normal in the first hour; if it never rises, compare the <code>[eval]</code> lines with <code>reports/logs/rl/run1/eval.jsonl</code> and check the reward change you made

Appendix A - Phase 0 report

REPORTS/PHASE_0.MD

Phase 0 — Environment audit and setup

Status: PASSED

Acceptance criteria

- [PASS] One SIH drone + one MAVROS instance: script arms, takes off to 10 m, hovers 20 s, lands. Evidence: `scripts/phase0_flight_test.py`, run twice, both "result": "PASS":
- run 1: `reports/logs/phase_0/run1/flight_summary.json`, `flight_altitude.csv` (518 rows, 10 Hz)
- cross-check run (after `pkill`, from a clean `env -i` shell): `reports/logs/phase_0/run2_crosscheck/flight_summary.json`, `flight_altitude.csv`
- plot of altitude over time: `reports/phase0_altitude.png`
- PX4 console confirms `Landing detected / Disarmed by landing (run*/px4_1.log)`
- [PASS] Peak RAM for 1 drone (PX4 + MAVROS) measured and recorded: **135.0 MB** (PX4 10.1 MB + MAVROS 124.9 MB), kernel peak RSS (VmHWM from `/proc/<pid>/status`), matching the 10 Hz sampled peak exactly. Evidence: `run2_crosscheck/flight_summary.json` (`peak_rss_vmhwm_mb`, `peak_rss_sampled_mb`). Run 1 (sampled only): 135.2 MB.

Phase 0 tasks: - [DONE] Environment recorded: `reports/logs/phase_0/env_audit.txt`, `env_audit_crosscheck.txt`. - [SKIPPED — user's request] ntfy test. The user said not to bother with phone notifications. `scripts/notify.sh` exits 0 silently because `~/.config/swarm-ntfy/topic` does not exist. - [DONE] Installed PX4 toolchain (`ubuntu.sh --no-nuttx --no-sim-tools`), `ros-jazzy-mavros` 2.15.1, GeographicLib datasets: `sudo_installs.log`, `geographiclib_datasets.txt`. - [DONE] Built `px4_sitl_sih` with 2 jobs: `px4_build.log` (`BUILD_EXIT=0`, [784/784]). - [DONE] Repo skeleton: `config/`, `src/swarm_agent/`, `src/swarm_tools/`, `tests/`, `scripts/`, `reports/`.

Key numbers

Item	Value	Source
OS	Ubuntu 24.04.5 LTS, kernel 7.0.0-34-generic, x86_64	<code>env_audit.txt</code>
ROS 2	Jazzy (<code>ROS_DISTRO=jazzy</code>)	<code>env_audit.txt</code>
Python	3.12.3	<code>env_audit.txt</code>
RAM	7721 MiB (8,097 MB) total; available 2357–3150 MiB (<code>free -m</code>) / 3,267–3,287 MB (<code>psutil</code>) during this phase, with Chrome open (~3.1 GB)	<code>env_audit*.txt</code> , <code>flight_summary.json</code>
CPU	Intel i3-1115G4, 2 cores / 4 threads	<code>env_audit.txt</code>
Free disk	131 GB before, 128 GB after the PX4 build (need ≥ 15 GB)	<code>env_audit.txt</code> , <code>env_audit_crosscheck.txt</code>
PX4	v1.18.0-rc1, target <code>px4_sitl_sih</code> , model <code>sihsim_quadx</code>	<code>env_audit_crosscheck.txt</code>
MAVROS	2.15.1 (apt)	<code>env_audit_crosscheck.txt</code>
PX4 build (resumed run)	2 min 12 s wall; largest single process 383,512 kB (≈ 393 MB); <code>ninja -j2</code>	<code>px4_build.log</code>

Flight test	Run 1	Cross-check
MAVROS connected after launch	1.3 s	1.3 s
OFFBOARD + armed	2.0 s	1.1 s
Climb 0 → 10 m (±0.3 m)	10.3 s	10.3 s
Hover duration	20.0 s	20.0 s
Hover altitude mean / min / max	10.018 / 9.723 / 10.116 m	9.943 / 9.706 / 10.125 m
AUTO.LAND → disarmed	20.7 s	19.6 s
Final landed_state	1 (ON_GROUND)	1 (ON_GROUND)
Peak RAM PX4 + MAVROS	135.2 MB (sampled)	135.0 MB (VmHWM)

Sources verified (URLs / commands)

- Build target `px4_sitl_sih`, run `make px4_sitl_sih sihsim_quadx`, multi-instance `sitl_multiple_run.sh`, `PX4_HOME_LAT/LON/ALT`: https://docs.px4.io/main/en/sim_sih/
- `boards/px4/sitl/sih.px4board` exists at tag v1.18.0-rc1 and `main`, but **not** at v1.17.0 (latest stable) or v1.16.2 (GitHub raw checks returned 404 at those two tags).
- Ubuntu setup `ubuntu.sh [--no-nuttx] [--no-sim-tools]`, 24.04 supported: https://docs.px4.io/main/en/dev_setup/dev_env_linux_ubuntu. The script was read before running: the `dialout` group change sits inside the NuttX block (lines 126–197), so `--no-nuttx` skips it. A pip dry run showed 16 new packages and **no upgrades** (numpy stays 1.26.4).
- MAVROS install + `install_geographiclib_datasets.sh` + `px4.launch`: <https://github.com/mavlink/mavros/blob/ros2/mavros/README.md>; `/opt/ros/jazzy/share/mavros/launch/{px4.launch,node.launch,px4_config.yaml,px4_pluginlists.yaml}`.
- Offboard: setpoints must stream ≥ 2 Hz before switching into OFFBOARD and while in it (`COM_OF_LOSS_T`, `COM_OBL_RC_ACT`): https://docs.px4.io/main/en/flight_modes/offboard
- Installed PX4 scripts (`build/px4_sitl_sih/etc/init.d-posix/`):
- `px4-rc.mavlink`: offboard link of instance *n* listens on UDP 14580+*n* and sends to 14540+*n* (instances > 9 share 14549)
- `rcS`: `MAV_SYS_ID = instance + 1`; `PX4_SIM_MODEL` selects the airframe by name

- `px4-rc.sihsim: PX4_HOME_* → SIH_LOC_LAT0/LON0/H0`
- `airframes/10040_sihsim_quadx: SIH quad`
- `rcS: battery_simulator` starts when `SIM_BAT_DRAIN > 0` (for Phase 4 F3)
- PX4 Makefile lines 77–107: `-jN` is read from `ps T` (needs a terminal); `j=2` sets it explicitly.
- MAVROS interfaces seen on the running system: `ros2 topic list`, `ros2 service list (/uav1/mavros/state, local_position/pose, global_position/rel_alt, extended_state, setpoint_velocity/cmd_vel, cmd/arming, set_mode)`, plus the message/service definitions (`mavros_interfaces.txt`, `/opt/ros/jazzy/share/mavros_msgs/msg/State.msg`).

Problems and fixes

1. **The build target in docs/SPECIFICATION.md is missing from stable PX4.** `px4_sitl_sih` is absent from v1.17.0, so PX4 is pinned to **v1.18.0-rc1** (`config/swarm.yaml: sim.px4_version`).
2. **GeographicLib geoid and magnetic downloads failed.** The SourceForge mirror `yer.dl.sourceforge.net` failed its SSL handshake, and the MAVROS script hides errors and still exits 0. Fix: re-downloaded with the same tools (`geographiclib-get-geoids/-magnetic`, retry on a working mirror) into scratch space, then `sudo cp` into `/usr/share/GeographicLib`.
3. **The first build ran 6 parallel jobs**, breaking rule 6. PX4's Makefile only detects `-j2` from a terminal. It was stopped at 344/1160 and resumed with `make px4_sitl_sih j=2` (`scripts/build_px4.sh`); `ninja -j2` confirmed in `ps`.
4. **PX4's startup script breaks on paths with spaces.** The repo path contains one ("drone swarm") and the error was `no autostart file found`. PX4 working dirs now go to `sim.work_dir (/tmp/swarm_sim/<timestamp>/px4_<id>)`: a fresh dir each run, never deleted by our tools. Console logs stay in the repo.
5. **MAVROS aborts on an empty parameter override** (`-p gcs_url:=`). The empty value is the default anyway, so the argument was removed.
6. `ros2 interface show mavros_msgs/msg/State` **printed ExtendedState's fields.** The authoritative `.msg` file and a live `ros2 topic echo` both show the correct State fields.
7. `kill -f px4` (**rule 3a**) **also kills the shell that runs it**, because its own command line contains "px4". Replaced with exact-name matching (`kill -x px4; pkill -x mavros_node`, and `kill_orphans()` in `src/swarm_tools/sim_launch.py`).

Known limitations / honest caveats

- PX4 v1.18.0-rc1 is a release candidate. Real Pixhawk 6C drones should run the same version when Phase 6 mixed-reality tests happen.
- Two flight runs only; the evidence is for a single drone.
- After auto-disarm, `mavros/state.mode` reads `OFFBOARD` again, because the test still streams setpoints. The vehicle is disarmed on the ground; harmless, noted for Phase 3.
- The takeoff used `OFFBOARD` velocity setpoints (the pipeline Phases 3–6 need), not `AUTO.TAKEOFF`.
- PX4 writes `.ulg` flight logs into `/tmp/swarm_sim/...`; the OS cleans `/tmp`, our tools don't.
- CPU load was not measured in Phase 0 (not required); Phase 1 measures it.

Next phase: what is needed from the user

Nothing. All sudo installs are done. For Phases 1 and 3–5, closing Chrome during multi-drone runs frees about 3 GB of RAM.

Appendix B - Phase 1 report

REPORTS/PHASE_1.MD

Phase 1 — Multi-drone simulation scaling

Status: PASSED — max stable N on this laptop = **10** (the target), with a lean MAVROS plugin list

Acceptance criteria

- [PASS] All N drones arm, take off to 10 m, hover 30 s, and land, with no crashes, for N = 3, 5 and 10. Each N ran twice: once, then again from a clean `env -i` shell after `kill -x px4`; `kill -x mavros_node`. Evidence: `reports/logs/phase_1/n{3,5,10}/summary.json` and `n{3,5,10}_crosscheck/summary.json`, all "result": "PASS", with every criterion true: `all_armed_took_off_to_10m`, `hover_30s`, `all_landed_disarmed` (landed_state 1 for every drone), `no_process_crash` (no PX4/MAVROS process exited before shutdown), `distinct_spawn_positions`. Per-drone altitude traces: `n*/altitude.csv`.
- [PASS] Report of max stable N with RAM/CPU numbers: **N = 10**, 1,074 MB peak RAM for PX4 + MAVROS (107.4 MB per drone), 76.5 % mean system CPU during hover (cross-check run). N = 10 is also the most the PX4 SITL port plan supports unchanged (`px4-rc.mavlink`: `instances > 9` share UDP port 14549).

Phase 1 tasks: - [DONE] Launch script for N SIH + N MAVROS instances, each with unique ports and sysid: `src/swarm_tools/sim_launch.py`. PX4 instance `n` gets `-i n`; its offboard link is UDP 14580+`n` ↔ 14540+`n`; `MAV_SYS_ID = n+1`; MAVROS runs in namespace `/uav<id>` with `fcu_url udp://:14540+n@127.0.0.1:14580+n` and `tgt_system n+1`. Serial URLs are refused. - [DONE] Verified spawn positions. Without `PX4_HOME_*`, two instances spawn at the same point (0.02 m apart, PX4 default home 47.3977427, 8.5455942): `spawn_default_check.txt`. Fix: each drone gets its own home via `PX4_HOME_LAT/LON/ALT`, the documented method (`docs.px4.io/main/en/sim_sih`; mapped to `SIH_LOC_LAT0/LON0/H0` in `px4-rc.sihsim`). The layout is the V formation itself (`formation.initial_layout`, 10 m spacing, apex = lowest ID, facing the goal), so drones take off already in their slots. Measured against plan, from the GPS fix converted to the shared ENU frame: max error 0.19 m, min pair distance 9.75 m. - [DONE] Scaled 3 → 5 → 10, measuring RAM (kernel VmHWM per process) and CPU (psutil, 1 Hz) at each step: `n*/resources.csv`. Plot: `reports/phase1_resources.png`.

Key numbers

Idle laptop, no sim (Opera ≈ 31 % of a core, gnome-shell ≈ 12 %): **14.6 %** system CPU, 3,253 MB available (`idle_baseline.txt`). System CPU is % of all 4 logical CPUs; per-process CPU is % of one core. RAM is decimal MB.

Run	MAVROS plugins	Result	RAM total (MB)	RAM / drone	System CPU hover mean / max	PX4 CPU (all)	MAVROS CPU (all)	Harness CPU	Min available RAM
n3_default_mavros	default	PASS	513.7	171.2	44.1 / 46.7 %	25.0 %	84.7 %	37.4 %	2,867 MB
n5_default_mavros	default	PASS	987.2	197.4	99.7 / 99.9 %	37.7 %	281.4 %	42.8 %	2,650 MB
n3	lean	PASS	288.1	96.0	25.0 / 42.0 %	23.7 %	23.0 %	27.9 %	3,609 MB
n5	lean	PASS	496.4	99.3	41.2 / 45.7 %	38.9 %	44.6 %	52.1 %	3,516 MB
n10	lean	PASS	1,070.9	107.1	76.1 / 77.2 %	72.4 %	116.0 %	82.1 %	3,203 MB
n3_crosscheck	lean	PASS	287.6	95.9	24.9 / 26.3 %	24.2 %	22.8 %	28.0 %	3,766 MB
n5_crosscheck	lean	PASS	496.3	99.3	41.0 / 46.0 %	39.3 %	45.0 %	51.4 %	3,522 MB
n10_crosscheck	lean	PASS	1,074.2	107.4	76.5 / 77.4 %	71.4 %	121.8 %	78.7 %	3,477 MB

Run	Spawn error max	Spawn min pair	Hover min pair	Arm → all at 10 m	Land → all disarmed
n3 / n3_crosscheck	0.10 / 0.11 m	10.09 / 10.08 m	9.18 / 9.13 m	12.3 / 12.2 s	22.4 / 20.3 s
n5 / n5_crosscheck	0.19 / 0.19 m	10.06 / 10.10 m	9.00 / 8.94 m	13.3 / 12.5 s	23.5 / 21.2 s
n10 / n10_crosscheck	0.17 / 0.16 m	9.75 / 9.76 m	8.32 / 9.05 m	13.1 / 13.2 s	23.6 / 25.0 s

Idle MAVROS scaling (`idle_scaling_*.jsonl`; drones connected, on the ground, 15 s window):

N	Default: MAVROS CPU per instance	Default: RSS per instance	Lean: MAVROS CPU per instance	Lean: RSS per instance
1	17.8 %	123.7 MB	4.1 %	81.9 MB
3	36.6 %	159.3 MB	4.6 %	85.2 MB
5	65.7 %	185.5 MB	4.7 %	88.3 MB

Sources verified (URLs / commands)

- Multi-instance SIH: `./Tools/simulation/sitl_multiple_run.sh 3 sihsim_quadx px4_sitl_sih` and manual `px4 -i <n> -d <build>/etc`: `https://docs.px4.io/main/en/sim_sih/` and the script itself at v1.18.0-rc1 (one working dir per instance).
- Distinct home per instance, `PX4_HOME_LAT/LON/ALT`: `https://docs.px4.io/main/en/sim_sih/`; `build/px4_sitl_sih/etc/init.d-posix/px4-rc.sihsim`.
- Ports and sysid: `px4-rc.mavlink` (14580+`n` local, 14540+`n` remote, instances > 9 → 14549); `rcS` (`MAV_SYS_ID = px4_instance + 1`).
- MAVROS plugin names: `/opt/ros/jazzy/share/mavros/mavros_plugins.xml`. Allow/deny list parameters: the commented `plugin_allowlist` in `mavros/launch/px4_pluginlists.yaml`. With the lean list, every needed topic and service was checked on a running system: `lean_mavros_interfaces.txt`.
- Shared topics found with `ros2 topic list / ros2 topic info` (3 drones, default plugins): `/parameter_events` had 82 publishers and 84 subscribers; `/tf` and `/tf_static` had 3 and 3.
- Discovery range: installed `librcl.so` says "ROS_LOCALHOST_ONLY is deprecated ... Use ROS_AUTOMATIC_DISCOVERY_RANGE", and `rmw/discovery_options.h` lists `LOCALHOST / SUBNET / OFF`. `scripts/ros_env.sh` sets `ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST`. `docs.ros.org` refused the automated fetch (bot-protection page), so the installed code was used.

Problems and fixes

1. **Default MAVROS scaled $O(N^2)$** , so 5 drones already saturated the CPU (99.7 %). Every MAVROS instance runs about 27 ROS nodes (one per plugin). Each publishes to and subscribes to the global `/parameter_events`, and the `param` plugin mirrors about 1,000 PX4 parameters into ROS parameters. Result: per-instance CPU climbed with N even while idle (17.8 → 36.6 → 65.7 %). Fix, still MAVROS (no architecture change): - `config/mavros_pluginlists.yaml` loads only the 6 plugins an agent needs (`sys_status`, `sys_time`, `command`, `local_position`, `global_position`, `setpoint_velocity`) - `/tf`, `/tf_static` and `/parameter_events` are remapped into each drone's namespace Per-instance CPU is now flat at 4.1–4.7 %, and N = 5 hover dropped to 41 % system CPU.
2. **PX4 default spawn is identical for all instances** (0.02 m apart). Fixed with per-drone `PX4_HOME_*` as documented (see above).
3. **rclpy prints "The following exception was never retrieved: cannot use Destroyable..."** at shutdown. These are fire-and-forget service calls still pending when the node is destroyed; cosmetic, after the test has finished.

Known limitations / honest caveats

- **CPU is the constraint at N = 10:** 76.5 % of all 4 logical CPUs during hover. That includes the Python test harness (~80 % of one core) and about 15 % background (Opera, desktop). Phase 3 replaces the harness with 10 agent nodes, a link emulator and a logger. Their CPU is unmeasured yet and could push the laptop to saturation. Closing Opera/Chrome frees about 15 %.
- The 1-minute load average (12.8 mean at N = 10, up to 29 during startup) overstates pressure: MAVROS has 22 threads per instance, and load averages carry over between back-to-back runs. CPU % is the better measure.
- The lean plugin list drops `param`, `home_position`, `imu`, `rc_io` and others. PX4 parameters for fault injection (e.g. `SIM_BAT_DRAIN`) will be set another way, to be verified in Phase 4.
- Spawn check uses SIH's simulated GPS (noise ≈ 0.1 – 0.2 m). Hover-time distances use the GPS fix horizontally and local z vertically; all homes share one ground altitude.
- Timing numbers are wall-clock; PX4 runs lockstep SIH. No sign of sim slowdown at N = 10 (arm → all at 10 m took 13.1 s, vs 12.3 s at N = 3).

Next phase: what is needed from the user

Nothing. If CPU becomes the limit in Phase 3 at N = 10, closing the browsers during runs is the first option before reducing N.

Appendix C - Phase 2 report

REPORTS/PHASE_2.MD

Phase 2 — Agent logic (pure Python, no ROS) + unit tests

Status: PASSED

Acceptance criteria

- [PASS] `pytest` all green: **91 passed** (`reports/logs/phase_2/pytest_coverage.txt`; also green in the clean-shell cross-check).
- [PASS] Coverage $\geq 90\%$ on the election and formation modules: `election.py` **99 %** (167 statements, lines 118–119 not hit), `formation.py` **100 %** (57 statements). All logic modules are $\geq 98\%$; 99 % in total (`pytest --cov`, same file).
- [PASS] 1,000 randomized pure-Python runs (random kills, random link drops, partitions): exactly one master after convergence in **1000 / 1000 (100 %)** runs. **Median convergence 0.145 s** after the last fault cleared (p95 2.12 s, max 2.72 s). Evidence: `reports/logs/phase_2/random_trials_summary.json` and `random_trials.jsonl` (one line per run: seed, N, latency, loss, applied faults, convergence, masters). The clean-shell cross-check (`crosscheck/`) reproduced the per-run file byte for byte.

What was built (all importable and testable without ROS)

Module	Role
<code>src/swarm_agent/geometry.py</code>	WGS-84 $\leftrightarrow$ ECEF $\leftrightarrow$ one shared ENU frame at <code>config.origin</code> ; tuple vector helpers
<code>src/swarm_agent/heartbeat.py</code>	Heartbeat dataclass + 50-byte binary codec (radio-sized)
<code>src/swarm_agent/election.py</code>	Election state machine: term, timeouts, claims, step-down, planned handover
<code>src/swarm_agent/formation.py</code>	V slots, slot assignment, rotation into master heading, follower control law
<code>src/swarm_agent/safety.py</code>	Pairwise separation, repulsion, geofence
<code>src/swarm_agent/agent_core.py</code>	One drone's full logic: mission phases + election + formation + safety; the Phase 3 ROS node wraps it unchanged
<code>src/swarm_agent/config.py</code>	Typed, strict loader for <code>config/swarm.yaml</code> with cross-field validation
<code>src/swarm_tools/puresim.py</code>	Point-mass simulator running the real <code>AgentCore</code> per drone; lossy/delayed/blockable heartbeat network
<code>scripts/random_trials.py</code>	The 1,000-run acceptance experiment
<code>scripts/puresim_faults.py</code>	F1–F5 of Phase 4 in the pure simulator (design check before PX4)

Key numbers

Randomized runs (`random_trials_summary.json`): - **Setup per run**: N uniform in 3..10 (121–132 runs each), link latency {0, 50, 150, 300} ms (+20 % jitter), random loss 0–30 %, and 1–5 random faults. Faults applied: `kill_master` 454, `kill_random` 470, `master_link_lost` 499, `link_drops` 494, `partition` 468, `low_battery_master` 397. - **Wall time**: 100.4 s on 3 worker processes. - **Median convergence by the run's last fault**: `kill_master` 1.73 s (a true failover), `partition` 0.29 s, `link_drops` 0.20 s, `low_battery_master` 0.18 s, `kill_random` 0.03 s, `master_link_lost` 0.03 s. - **Split brain during faults**: 801 runs had more than one master at some point. That is expected: both sides of a partition elect a leader. Every run still ended with exactly one.

F1–F5 in the pure simulator, N = 10, 20 seeds each, ideal links (`puresim_faults_n10.txt/.jsonl`). This checks the design before PX4, not Phase 4 evidence:

Fault	Handover (median / worst)	Formation RMS < 2 m after (median / worst)	Min separation	Goal reached
F1 master killed	1.64 / 1.75 s	4.55 / 4.68 s	9.35 m	20/20
F2 master link lost	1.64 / 1.75 s	4.55 / 4.68 s	8.23 m	20/20
F3 master low battery	0.17 / 0.20 s (planned)	3.14 / 3.23 s	9.35 m	20/20; retiree lands ≤ 2.3 m from home
F4 follower killed	no master change	1.12 / 1.15 s	7.46 m	20/20
F5 partition, then heal	0.18 / 0.25 s after heal	9.93 / 13.75 s	6.23 m	20/20

Design decisions that complete the spec (all tunables in `config/swarm.yaml`)

- Heartbeat fields beyond §5**. In addition to id, role, term, position, velocity, battery and timestamp, the heartbeat carries: - `phase`, the mission phase; followers copy the master's - `master_id` - `handover_to` - `flags`: MASTER_OK, READY, ELIGIBLE, ORPHAN, AIRBORNE - the master's formation `heading` - `members`, a bitmask of the followers the master hears

Position and velocity are in the shared ENU frame; each drone converts its own lat/lon. Encoded size is 50 bytes, so 10 drones at 5 Hz is about 2.5 kB/s per receiver. 2. **Election** (`election.py` docstring). Lowest-ID alive eligible drone claims with term = highest seen + 1; the step-down rule is exactly §6. Additions: - (a) a drone never claims while any peer still reports hearing a live master (MASTER_OK). This prevents failovers caused by one drone's own bad link. - (b) it waits while a lower-ID eligible drone is alive. - (c) no preemption: a recovered lower ID does not take leadership back. - (d) nobody claims during the first 3 s after boot. - (e) planned low-battery handover: the master names the lowest-ID eligible peer and keeps leading until that drone's higher-term heartbeat arrives. If nobody takes over within 1 s, it names the next candidate. 3. **Slots come from the master's member list**, so every follower computes the same assignment even when their own views differ. A drone the master cannot hear becomes an **orphan**: after missing from the list for 1 s it flies on a layer 8 m above cruise. 4. **Rank $\rightarrow$ V slot mapping**. Arm = ID parity (even left, odd right); row = rank by ID within the arm; arms rebalance only when their lengths differ by more than 2. This keeps §6 "slot = rank among alive drones sorted by ID" while avoiding a failure of the plain alternating mapping: - With alternating left/right ranks, losing one drone makes every higher rank swap sides. With 10 drones, F1 then requires lateral moves up to 64 m across the formation. - With parity arms, F4 only slides the same-arm drones behind the lost one by one slot. F1 keeps the new master's arm fixed and shifts the other arm by exactly 10 m. Both are unit-tested in `test_formation.py`. 5. **Transit layer**. A follower whose slot is more than 12 m away horizontally (for ≥ 0.5 s) flies to it on a layer 6 m below the master. It moves sideways at most 1 m/s until it is 5 m (min separation) clear of the formation layer, and returns once within 4 m of the slot. This makes the large re-anchoring after a partition heals

safe: a new master is often behind the rest. 6. **Altitude layers** relative to cruise (30 m): formation 0, transit -6, orphan +8, returning (low battery) +15 m. Validation requires layers ≥ 5 m apart and the top one below the 50 m fence minus a 3 m margin. 7. **Low battery ($\leq 30\%$)**. The drone keeps the formation's horizontal velocity while climbing to the +15 m layer, flies home there at cruise speed, then lands. 8. **Ghost obstacles**. A drone keeps avoiding a lost peer's position, extrapolated from its last heartbeat, for 3 s. 9. **Mission**. IDLE $\rightarrow$ TAKEOFF (all expected drones READY, or a 30 s timeout) $\rightarrow$ CRUISE (every member within 1.5 m of 30 m) $\rightarrow$ HOLD (master within 3 m of the goal; hover 10 s) $\rightarrow$ LAND. Followers copy the phase. A new master restarts the current phase timer. 10. **Spawn layout = the V itself** (Phase 1), so takeoff needs no reshuffle. 11. **Vertical rates** follow PX4 defaults verified in source: descent 1.5 m/s (`MPC_Z_VEL_MAX_DN`), ascent ≤ 3 m/s (commanded 2), horizontal ≤ 12 m/s (commanded ≤ 10).

Sources verified (URLs / commands)

- PX4 v1.18.0-rc1 defaults, read from `src/modules/mc_pos_control/*_params.yaml`:
- `multicopter_position_control_limits_params.yaml`: `MPC_XY_VEL_MAX` 12.0, `MPC_Z_VEL_MAX_UP` 3.0, `MPC_Z_VEL_MAX_DN` 1.5
- `multicopter_autonomous_params.yaml`: `MPC_ACC_HOR` 3.0
- `multicopter_altitude_mode_params.yaml`: `MPC_ACC_UP_MAX` 4.0, `MPC_ACC_DOWN_MAX` 3.0
- `multicopter_takeoff_land_params.yaml`: `MPC_LAND_SPEED` 0.7
- WGS-84 constants and ECEF/ENU formulas are standard. The implementation is checked against an independent curvature-radius formula (1 km offsets within 5 cm, and the 0.085 m poleward drift of a parallel matches $d^2 \cdot \tan \varphi / 2R$ within 5 mm) and against ECEF distances (`test_geometry.py`).
- Phase 2 needs no new external interfaces (no ROS/MAVROS calls).

Problems and fixes

1. **Alternating slot mapping caused large cross-formation moves** and separation dips below 5 m in the F1/F5 simulations. Fixed with parity arms (decision 4).
2. **Partition heal: the winning master (higher term) is usually behind the formation**, so the front group had to fall back through the rear. Fixed with the transit layer (decision 5).
3. **Bugs found by simulation, and fixed**: - *Retiring master counted as a member*: during a handover the retiring master was briefly listed as a member, shifting slots. Fix: members = master + follower peers only. - *Descent capped by a 3-D saturation*: one saturation on the 3-D correction also capped the descent rate. Fix: horizontal and vertical are limited separately. - *Ping-pong at 25 m*: the "climb before joining" rule fought the transit layer. Fix: it now applies only to drones that never reached cruise altitude.
4. **Four wrong test expectations** were corrected; the code was right each time: - the poleward curvature of a parallel - the heartbeat is 50 bytes, not 49 - rebalancing moves two drones in that case - an orphan ranked behind another orphan

Known limitations / honest caveats

- **Separation when drones cannot hear each other**. 143 of the 1,000 randomized runs had a minimum separation below 5 m (worst 0.02 m), and **every one of them contained a random partition or random link drops**. The 362 runs without those faults (kills, master link loss, low battery; loss up to 30 %, latency up to 300 ms, up to 5 faults) had **none**. Among single-fault runs: partition 5/33, link_drops 1/28, all other fault types 0. Randomly chosen groups are spatially interleaved; each group compacts its formation into space the other group occupies, and repulsion cannot act on drones that can't be heard. Phase 4 F5 will use **spatially separated groups** (the lower-ID front of the V vs the higher-ID back). That is what a distance-driven radio partition looks like, and in that case the pure-sim minimum was 6.23 m. Protecting against interleaved blind pairs would need onboard proximity sensing or sticky formation references. That is not in this phase; left for user review.
- The pure simulator is a point mass: first-order velocity tracking, τ 0.35 s, acceleration limits 4 m/s² horizontal and 3 m/s² vertical. PX4 dynamics differ; Phase 3/4 measure the real behaviour.
- Convergence in the randomized runs is measured after all links are healed and random loss is switched off (latency stays). Behaviour under sustained loss is Phase 5.
- The design decisions above complete gaps in docs/SPECIFICATION.md. Decisions 3-5 (orphan layer, parity arms, transit layer) change how slots and altitudes behave during faults; the user should review them.

Next phase: what is needed from the user

Nothing to proceed. Please review the design decisions above, especially parity arms, the transit and orphan layers, and the F5 partition choice.

Appendix D - Phase 3 report

REPORTS/PHASE_3.MD

Phase 3 — ROS 2 integration: formation flight, no faults

Status: PASSED — all flight criteria met in every run at the maximum stable N (10 PX4 drones). Caveat: in 2 of the 6 recorded runs the test harness (not the drones) timed out; see below.

Acceptance criteria

- [PASS] All drones reach the goal and land. In all 6 runs every drone landed in its formation slot around the goal: the leader 1.1–1.7 m from the goal point, the last drone of each arm about 49 m behind it (its slot). Evidence: `reports/logs/phase_3/<run>/metrics.json` (`final*.ok`, `dist_to_target_m`).
- [PASS] Formation RMS error < 2 m during cruise: worst value per run 1.24–1.66 m (`formation_rms_cruise_m.max`), mean 0.54–0.62 m. Evidence: `metrics.json`, `formation_rms.csv`.
- [PASS] Minimum separation $\geq$ 5 m at all times: 7.80–8.65 m (`min_separation_m`).
- [PASS] Plot of trajectories saved to `reports/: reports/phase3_mission.png` (tracks with V snapshots, followers against their ideal slots, formation RMS over time).

Phase 3 tasks: - [DONE] `swarm_agent` node wrapping the Phase 2 modules: `src/swarm_agent/ros_node.py` (one per drone, namespace `/uavN`, its own MAVROS). - [DONE] `link_emulator` node with loss 0: `src/swarm_tools/link_emulator.py`. - [DONE] Launch file: `launch/swarm.launch.py` (driven by `scripts/run_mission.py`). - [DONE, with a note] Logger writing CSV: `src/swarm_tools/logger_node.py` writes JSON lines (`states.jsonl`, one sample per drone at 10 Hz); `swarm_tools.logfmt.jsonl_to_csv` turns them into CSV with fixed columns (`states.csv.gz` in every run folder). - [DONE] Metrics script: `scripts/metrics.py` (formation RMS, minimum separation, time to goal).

Key numbers

Run	Harness	Drones	RMS max / mean / p95 (m)	Min separation (m)	Time to goal (s)	Leader to goal (m)	Leaders at once
run1	completed	10	1.32 / 0.56 / 1.08	8.11	221.4	1.2	1
run2	timeout (flight complete)	10	1.39 / 0.62 / 1.13	8.65	220.4	1.3	1
run3	timeout (flight complete)	10	1.66 / 0.56 / 1.02	7.80	220.3	1.7	1
crosscheck1	completed	10	1.56 / 0.56 / 1.03	8.10	222.1	1.4	1
crosscheck2	completed	10	1.24 / 0.54 / 1.00	8.57	220.6	1.4	1
crosscheck3	completed	10	1.36 / 0.60 / 1.08	8.02	226.5	1.1	1

Mission: 1 km north of home, V formation, 10 m spacing, 30 m, 5 m/s, hover 10 s at the goal, land. The cross-check runs were started from a clean `env -i` shell after killing every PX4 and MAVROS process (`scripts/phase3_runs.sh`; console output in `reports/logs/phase_3/batch.out`).

Sources verified (URLs / commands)

- PX4 Offboard mode needs a continuous setpoint stream of at least 2 Hz and exits Offboard after `COM_OF_LOSS_T` without it (https://docs.px4.io/main/en/flight_modes/offboard.html); the agent streams velocity setpoints at 20 Hz (`config/swarm.yaml` `setpoints.rate_hz`).
- MAVROS topics and services used by the node (`state`, `extended_state`, `global_position/global`, `local_position/pose` and `velocity_local`, `battery`, `setpoint_velocity/cmd_vel`, `set_mode`, `arming`, `set_message_interval`) were checked on a running system in Phase 1: `reports/logs/phase_1/lean_mavros_interfaces.txt`.
- MAVLink message IDs for the stream-rate requests were checked against pymavlink (`src/swarm_agent/ros_node.py`, `MAVLINK_MSG_IDS`; `tests/test_ros_constants.py`).

Problems and fixes

- Formation RMS spiked at the start of cruise: the leader now ramps its speed (0.5 m/s²), so followers that see its velocity only at the heartbeat rate keep up.
- CPU saturation with 10 drones: MAVROS stream rates are set explicitly (position 20 Hz, others 1–2 Hz) and the link emulator's delivery timer runs only when latency or jitter is set.
- The leader wandered off the line to the goal: leg guidance (fixed heading + cross-track correction).
- A steady follower error: position gain raised to 1.0.
- Drone 3 sometimes became leader at startup: the first election now waits until the whole fleet is heard (or a timeout).
- run2 and run3 ended with the harness's watcher thread stalled (`InvalidHandle` in the readiness check). The flights themselves completed (every drone landed, logs complete); the other four runs (run1 and the three cross-checks) completed normally. The readiness check has since been given its own executor and a stall is now reported as `WATCHER_STALLED`; that change has not been re-run on PX4.

Known limitations / honest caveats

- PX4 SIH has a simple aircraft model and no wind; real aircraft will show larger errors.
- 10 drones is this laptop's limit for PX4 (Phase 1).
- No faults here by design; failover on PX4 is Phase 4 (not run yet).

Next phase: what is needed from the user

Phase 4 (fault trials on PX4, 10 trials $\times$ 5 faults) takes about 5 hours of laptop time; the scripts are ready (`scripts/faults.py`, `scripts/phase4_runs.sh`).

Appendix E - Phase 4 report (not completed)

REPORTS/PHASE_4.MD

Phase 4 — Failover under faults

Status: NOT COMPLETED — stopped by the owner on 27 September 2026 because this laptop cannot run the 55-trial PX4 batch reliably. The tools are ready and checked; this report says what was measured so far (indicative only), why the runs were stopped, and exactly how to run Phase 4 later.

In short. In Phase 4, ten PX4 drones fly the mission while one of five faults is injected (leader killed, leader's radio lost, leader's battery low, a follower killed, radio network split). Each fault had to be run 10 times (about 7 hours). The first attempt ran on battery (power-saver mode, CPU at 100 %) until the battery ran out; the second, on the charger, still overloaded the laptop and was stopped by the owner. So far: a new leader takes over in about 1.6–2.2 s (limit 3 s), the planned battery handover in 0.35–0.80 s (limit 1 s), and no two drones came closer than 5 m. Only F5 (after the radio split heals) needed about 17 s to reform (limit 15 s). How to run it later is in "How to run Phase 4 later" below.

What Phase 4 must show (docs/SPECIFICATION.md , section 7)

Faults, each at a random time during cruise, 10 trials each, 10 drones: - F1 master killed (its PX4 + MAVROS processes killed) - F2 master link lost (the link emulator drops 100 % of the master's heartbeats) - F3 master low battery (PX4's simulated battery if verifiable, else injected at the agent level) - F4 follower killed - F5 partition then heal (two groups, then reconnect)

Acceptance: - F1/F2: new master heartbeat within 3.0 s median (4.0 s worst case) of the failure. - F3: planned handover within 1.0 s of the trigger; the old master leaves and returns home. - F4: no master change; the formation recovers. - F5: a single master within 3.0 s after the heal. - All: formation RMS back under 2 m within 15 s; minimum separation never below 5 m; goal reached in at least 9 of 10 trials per fault. - Report table per fault: median / worst handover time, recovery time, minimum separation, success rate.

Acceptance criteria

None of them can be marked PASS or FAIL: the required 10 valid trials per fault were not run. What the partial runs indicate (conditions below): - [NOT MEASURED] F1/F2 handover — indicative: 1.62–2.23 s in all 6 F1/F2 trials (limit 3.0 / 4.0 s). - [NOT MEASURED] F3 planned handover — indicative: 0.35 and 0.80 s (limit 1.0 s); the old master landed at home both times. - [NOT MEASURED] F4 no master change — indicative: 2 of 2 without a change; formation back in 7.4–8.5 s. - [NOT MEASURED] F5 single master after the heal — indicative: 0.30 and 1.55 s (limit 3.0 s). - [NOT MEASURED] Formation under 2 m within 15 s — indicative: yes for F1–F4 (5.7–8.5 s); **no for F5** (16.7 and 17.1 s, see "Why F5 is slow"). - [NOT MEASURED] Minimum separation >= 5 m — indicative: 6.11 m or more in every trial. - [NOT MEASURED] Goal reached >= 9/10 per fault — indicative: every trial that ran to the end reached the goal (10 of 10).

What happened (27 September 2026)

Time	What	Outcome
12:31 – 13:57	First attempt: rounds 1–3 of F1–F5	The laptop was on battery in the power-saver profile (CPU about 1.7 GHz instead of up to 4.1 GHz). With 10 drones the CPU was at 99.9 % for the whole run (load average about 60; Phase 3 ran at about 17 and 67 % CPU on the charger). 10 trials completed; F1_t3 stopped when the whole machine stalled for 18 s; the battery ran out at 13:57 (the system log ends without a shutdown) and cut F2_t3 off. Kept apart: reports/logs/phase_4_on_battery/ (README there).
14:26	Batch restarted with a charger check	It waited for the charger and a non-power-saver profile.
14:50	Charger connected, Performance profile	The first trial started with a wrong fault name (a variable clash in the new charger check) and stopped at once; fixed in a31f1f0.
14:52 – 14:57	Second attempt, on the charger	F1 trial 1 flew: new leader after 1.62 s, formation back under 2 m after 5.71 s, closest pair 7.90 m. The load average still rose from 19 to about 40.
14:58	Stopped by the owner	"This laptop can't handle it." The trial was in the hover at the goal, not yet landed. reports/logs/phase_4/F1_t1/.

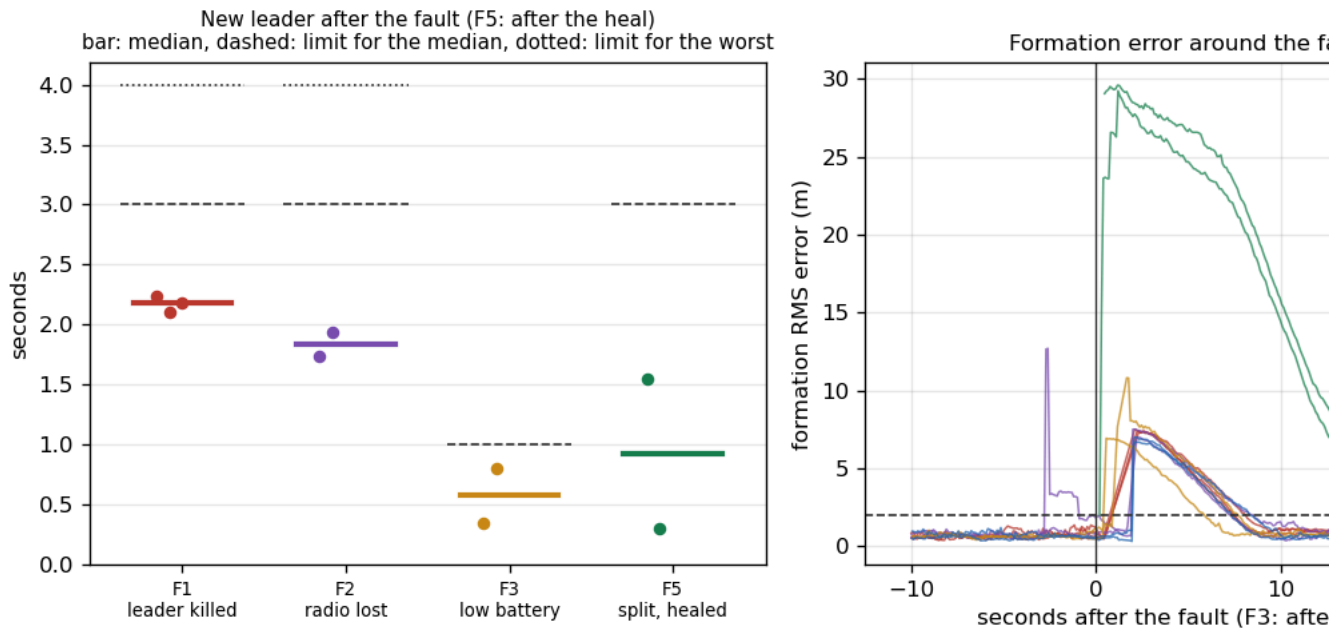
Key numbers so far (indicative only)

First attempt, on battery with a saturated CPU (reports/logs/phase_4_on_battery/phase4_on_battery_table.md , made by scripts/phase4_metrics.py from the trial logs):

Fault	Trial	New leader median / worst (s)	Limit (s)	Formation < 2 m median / worst (s)	Recovered within 15 s	Closest pair (m)	Goal reached	Other
F1	3	2.19 / 2.23	3.0 / 4.0	7.54 / 8.11	3/3	7.41	2/3 (F1_t3 stopped by the machine stall)	
F2	2	1.84 / 1.94	3.0 / 4.0	7.92 / 8.51	2/2	8.00	2/2	
F3	2	0.57 / 0.80	1.0	6.88 / 7.78	2/2	7.17	2/2	old leader landed at home: 2/2
F4	2	no change	no change	7.90 / 8.45	2/2	7.96	2/2	leader changed: 0/2
F5	2	0.93 / 1.55	3.0 (after the heal)	16.89 / 17.08	0/2	6.11	2/2	

Second attempt, on the charger (`reports/logs/phase_4/phase4_stopped_table.json`): F1 trial 1 — new leader 1.62 s, formation back 5.71 s, closest pair 7.90 m, reached the goal hover. The same fault with the same seed on battery took 2.23 s and 7.54 s: the power-saver CPU made everything slower.

Per-trial rows, including a short 0.4 s formation-error spike in F4_t1 (a late message under the saturated CPU) and the 18 s stall in F1_t3: `reports/logs/phase_4_on_battery/phase4_on_battery_table.md`. Figure: `reports/phase4_on_battery.png` (takeover times against the limits; formation error around each fault).



Why F5 is slow, and the options (the owner decides)

The front group (drones 1–5) and the back group (6–10) are split for 10–20 s. The back group elects drone 6 (term 2). After the heal, drone 1 (term 1) hears a master with a higher term and steps down, exactly as `docs/SPECIFICATION.md` section 6 prescribes, so **all ten drones re-form around drone 6**. Timeline of F5_t1 after the heal (from `states.jsonl`): every follower is 20–32 m from its new slot; each moves on the "transit layer" (drops about 6 m below the formation, crosses, climbs back) so paths cannot cross at the same height. PX4 descends at about 1.5 m/s and the drones climb at 2 m/s, so: about 7 s descending (with little sideways progress: the drones only move sideways slowly until they are 5 m below), about 6 s crossing, about 4–5 s climbing back. The height offset counts in the 3-D formation error, so the error only falls under 2 m after the climb: about 17 s in total. In the fast simulator (faster vertical response) the same fault recovered in about 10 s.

Options (none applied): 1. Keep the rules; make the move faster: allow sideways motion from half the transit depth, and/or a faster descent limit (PX4 `MPC_Z_VEL_MAX_DN` and `mission.descent_rate_mps`). Expected: 3–5 s less; smaller vertical margin during the move. 2. After a merge, hand the lead back to the lowest ID with the existing planned handover (0.35–0.80 s in F3), so only the back group re-slots. Changes the election behaviour written in `docs/SPECIFICATION.md`, so it needs the owner's approval. 3. Accept a longer limit for F5 (for example 20 s).

How to run Phase 4 later

What the computer needs

- **A stronger computer is recommended.** On this laptop (Intel i3-1115G4, 2 cores / 4 threads, 8 GB) 10 PX4 drones + 10 MAVROS + 10 agents load the CPU to 60–100 % even on the charger; on battery the CPU saturates. A machine with 8 or more cores and 16 GB (a desktop or a cloud VM) with the same setup (`docs/RUNBOOK.md`, section 1: Ubuntu 24.04, ROS 2 Jazzy, MAVROS 2.15.1, PX4 v1.18.0-rc1 built with `scripts/build_px4.sh`) runs it comfortably.
- **Or on this laptop with fewer drones:** `N=5 bash scripts/phase4_all.sh 1`. `docs/SPECIFICATION.md` rule 6 allows reducing the drone count; say so in the report.
- Always: on the charger, power mode Balanced or Performance (the batch waits otherwise), lid open, at least 800 MB free memory (`free -m`; `run_mission.py` refuses to start below that), and the Swarm Control app, the 3-D view and other heavy programs closed.

Commands (from the repository root)

```
bash scripts/phase4_all.sh 1      # rounds 1-10 of F1-F5, then round 11 from a clean shell as the cross-check
```

About 8 minutes per trial: 50 trials plus 5 cross-check trials, about 7 hours. If it is stopped, run the same command again: finished trials are skipped. To check the plan without flying: `DRY_RUN=1 bash scripts/phase4_runs.sh 1 2 dry_test` (then delete `reports/logs/dry_test`).

After the batch:

```
python3 scripts/phase4_metrics.py --out reports/logs/phase_4/phase4_table.json --md reports/logs/phase_4/phase4_table.md
reports/logs/phase_4/F?_t*
python3 scripts/phase4_metrics.py --out reports/logs/phase_4_crosscheck/phase4_table.json --md reports/logs/phase_4_crosscheck/phase4_table.md
reports/logs/phase_4_crosscheck/F?_t*
python3 scripts/plot_phase4.py reports/logs/phase_4/phase4_table.json --out reports/phase4_faults.png
```

Then rewrite this report: the per-fault table from `phase4_table.md`, every acceptance line marked PASS or FAIL with its number and file, the cross-check round compared, commit, and tag `phase-4` if it passed.

Checklist for later. (1) Use a stronger computer (8 or more cores, 16 GB), or this laptop with 5 drones. (2) Charger in, power mode Balanced or Performance, lid open, the 3-D view and other heavy programs closed. (3) Run `bash scripts/phase4_all.sh 1` (5 drones: `N=5 bash scripts/phase4_all.sh 1`); about 7 hours. If it stops, run the same command again: finished trials are skipped. (4) Afterwards run the `phase4_metrics.py` and `plot_phase4.py` commands above and rewrite this report with PASS/FAIL.

What each trial writes (`reports/logs/phase_4/F<k>_t<round>/`)

File	Contents
<code>states.jsonl.gz</code>	every drone's state 10 times a second: position, velocity, role, term, phase, battery, <code>claim_t</code> (when it became master), <code>retire_t</code> (when its battery handover started)
<code>fault_events.jsonl</code>	cruise detected, the fault (time, master before, target), the heal (F5)
<code>link_events.jsonl</code>	link-emulator commands (block the master's radio, split, heal)
<code>metrics.json</code> , <code>formation_rms.csv</code>	from <code>scripts/metrics.py</code> : formation error, closest pair, leader changes, where every drone landed
<code>run_summary.json</code>	result (COMPLETED / TIMEOUT / WATCHER_STALLED / ERROR), memory, power state before and after, CPU and memory use
<code>resources.csv</code>	CPU and memory once a second, per process group
<code>proc_logs/</code>	PX4, MAVROS and launch console logs
<code>../F<k>_t<round>.out</code>	the trial's console output

How each number is measured (`scripts/phase4_metrics.py`)

- The fault time is when the injector started the fault (before the kill commands ran), so the kill time counts against the result.
- F1/F2 handover: fault → the first `claim_t` of another drone (a new master sends its heartbeat in the same control step in which it claims).
- F3 handover: the old master's `retire_t` (battery at 30 %) → the successor's `claim_t`; "home": the old master landed within 5 m of its home.
- F5: the heal → the first moment every live drone follows one single master.
- Formation recovery: from the fault (F3: the battery trigger; F5: the heal), measured from the worst formation error in the next 30 s, until the RMS is under 2 m and stays there for 5 s.
- Minimum separation and "goal reached": `scripts/metrics.py` (3-D distance between airborne drones; every drone still reporting at the end landed in its slot at the goal, a retired one at home).

How the faults are made (`scripts/faults.py`)

Each fault fires at a seeded random time 30–150 s after the master first reports CRUISE (the batch uses the round number as the seed, so a round can be repeated exactly). - F1 / F4: SIGKILL the drone's PX4 and MAVROS process groups, then its agent process. - F2: the link emulator blocks every heartbeat the master sends, until the end of the run. - F3: `px4-param --instance <n> set SIM_BAT_MIN_PCT 20` on the master's PX4. PX4's battery simulator drains the battery while armed (full discharge in `SIM_BAT_DRAIN = 60` s) down to `SIM_BAT_MIN_PCT` (default 50 %); at 20 % it passes our 30 % handover level about 20 s later. - F5: the link emulator splits IDs 1–5 from 6–10 (the front and the back of the V, as distance would), and heals after a random 10–20 s.

Files that make up Phase 4

File	Role
<code>scripts/phase4_all.sh</code>	the whole batch: rounds 1–10, then the clean-shell cross-check round 11
<code>scripts/phase4_runs.sh</code>	one or more rounds of F1–F5; waits for the charger; skips finished trials; <code>DRY_RUN</code>
<code>scripts/run_mission.py</code>	one mission: checks memory and power, starts PX4 + MAVROS, the ROS 2 launch, injects the fault, waits for the landing, computes the metrics
<code>scripts/faults.py</code>	the five faults
<code>scripts/phase4_metrics.py</code>	per-trial and per-fault numbers, JSON and Markdown tables
<code>scripts/plot_phase4.py</code>	the figure
<code>scripts/metrics.py</code>	formation error, separation, goal, per run
<code>src/swarm_tools/sim_launch.py</code>	PX4 SIH and MAVROS instances, kill a drone
<code>src/swarm_tools/link_emulator.py</code>	heartbeat forwarding with block, split and heal commands
<code>src/swarm_tools/logger_node.py</code>	writes <code>states.jsonl</code>
<code>src/swarm_tools/resources.py</code>	CPU / memory sampler and <code>power_state()</code>
<code>src/swarm_agent/ros_node.py</code>	the drone agent; publishes <code>claim_t</code> and <code>retire_t</code>
<code>launch/swarm.launch.py</code>	link emulator, logger and one agent per drone
<code>config/swarm.yaml</code>	every setting (spacing, speeds, handover at 30 %, heartbeat 5 Hz, dead after 1.5 s, transit layer)

Sources verified (commands / files)

- PX4 battery simulator (installed PX4 v1.18.0-rc1, ~/PX4-Autopilot): `src/modules/simulation/battery_simulator/battery_simulator_params.yaml` (`SIM_BAT_DRAIN` default 60 s, full discharge while armed; `SIM_BAT_MIN_PCT` default 50 %, "can be used to alter the battery level during SITL simulation on the fly"); `BatterySimulator.cpp` (drains only while armed, back to 100 % when disarmed); `ROMFS/px4fmu_common/init.d-posix/rcS` (`battery_simulator` start when `SIM_BAT_DRAIN > 0`). Seen working: `px4_param_out "SIM_BAT_MIN_PCT: curr: 50.0000 -> new: 20.0000"` in every F3 trial's `fault_events.jsonl`.
- PX4 shell client instance selection: `platforms/posix/src/px4/common/main.cpp` (`--instance <n>` must be the first argument); instance = drone id – 1 (`src/swarm_tools/sim_launch.py, instance_of`).

- Power state: `/sys/class/power_supply/*` (type, online, capacity, status), `powerprofilesctl get`, `/proc/cpuinfo`; the previous boot's end without a shutdown: `journalctl --list-boots`, `journalctl -b -1`.

Problems found and fixed

- The agents did not publish when they became master or started a battery handover, so no takeover time could be measured: `ros_node.py` now publishes `claim_t` and `retire_t` (8252d4c).
- The fault time was taken after the kill commands had run (about 0.7 s later), which made takeovers look faster: it is now the moment the fault started.
- F3 formation recovery was counted from the parameter change, about 20 s before the formation was disturbed: it now counts from the battery trigger, and every recovery from the worst error after the reference.
- Trials on battery saturate the CPU: the batch now waits for the charger and a Balanced/Performance profile, and every trial records its power state (94a1148); a variable clash in that check was fixed (a31f1f0).
- 16 MB of state log per trial: compressed after each trial (about 1.5 MB); the metric scripts read the compressed files.

Known limitations / honest caveats

- No valid 10-trial set exists: all numbers above are indicative. Of the 12 trials with results, 11 ran on battery with a saturated CPU (10 to the end, 1 stopped by the stall), which made takeovers and recovery slower (compare the charger trial); the 12th ran on the charger and was stopped in the hover at the goal.
- Two trials per fault cannot show a median or a worst case; F5's 17 s is consistent across both trials and explained by the transit layer, so it is likely to fail the 15 s limit as the system stands.
- The earlier Phase 3 harness stall came back once (F1_t3), this time caused by the whole machine stalling on battery, not by the harness.

Next phase: what is needed from the owner

1. Decide where to run Phase 4: a stronger computer (recommended), or this laptop with 5 drones.
2. Decide how F5 should be handled (options 1–3 above).
3. Phase 5 (radio realism: 50/150/300 ms $\times$ 0/10/30 % loss) and Phase 6 build on Phase 4 and have the same computing needs.

Appendix F - Scaling test

REPORTS/SCALING.MD

Scaling test — does the swarm logic work for 1 to 100 drones?

Status: PASSED in the fast simulator (not PX4). Date: 2026-09-26. Commit of the code under test: 2291af7.

What was run

`scripts/scale_test.py` flies the default 1 km mission (`config/swarm.yaml`, V formation, 10 m spacing, 30 m, 5 m/s) in the pure-Python simulator (`src/swarm_tools/puresim.py`), which runs the real agent code (`AgentCore`: election, formation, safety) for every drone on point-mass physics. For every $N \geq 2$ the leader (drone 1) is killed at $t = 90$ s, mid-cruise. Three seeds per N.

- Clean: `PYTHONPATH=src python3 scripts/scale_test.py --ns 1 2 3 5 10 20 50 100 --seeds 1 2 3` → `reports/logs/scaling/fastsim_scaling.jsonl` (24 runs)
- Harder radio: same with `--loss-pct 10 --latency-ms 150 --drift 0.15` (10 % of heartbeats lost per link, 150 ms delay + up to 30 ms jitter, 0.15 m/s Gauss-Markov velocity drift) for $N = 10, 50, 100$ → `reports/logs/scaling/fastsim_scaling_radio.jsonl` (9 runs)
- Machine: `reports/logs/scaling/machine.txt` (i3-1115G4, 8 GB, other apps open). Sim speed depends on load.

Definitions: *New leader agreed* = time from the kill until every alive drone follows the same new leader with the same term. *Formation < 2 m after* = time from the kill until formation RMS error is back below 2 m. *Closest pair* = minimum distance between any two airborne drones over the whole flight. *Landed* = mission completed, every flying drone landed, exactly one leader at the end. Values are median (min–max) of 3 runs.

Results — clean radio, no drift

N	New leader agreed (s)	Formation < 2 m after (s)	Closest pair (m)	Cruise RMS max (m)	Landed	Sim speed (× real time)	Heartbeat (B)	Radio load (kbit/s)
1	–	–	–	–	3/3	2459.6 (2321.3–2483.4)	44	1.8
2	1.60 (1.50–1.60)	–	10.00	0.39	3/3	1506.0 (1289.0–1654.6)	44	3.5
3	1.70 (1.60–1.70)	4.75 (4.65–4.75)	9.49	0.73	3/3	818.5 (622.6–848.8)	44	5.3
5	1.70 (1.60–1.70)	4.60 (4.50–4.60)	9.49	0.73	3/3	426.8 (416.8–439.6)	44	8.8
10	1.70 (1.60–1.75)	4.50 (4.40–4.55)	9.49	0.73	3/3	162.1 (160.9–162.2)	45	18.0
20	1.70 (1.70–1.75)	4.60 (4.60–4.65)	9.49	0.73	3/3	57.3 (56.8–57.6)	46	36.8
50	1.75 (1.70–1.75)	4.95 (4.90–4.95)	9.49	0.73	3/3	10.9 (10.8–11.3)	50	100.0
100	1.75 (1.70–1.75)	8.75 (8.70–8.75)	9.49	0.73	3/3	2.9 (2.8–2.9)	56	224.0

Results — 10 % loss, 150 ms delay, drift 0.15 m/s

N	New leader agreed (s)	Formation < 2 m after (s)	Closest pair (m)	Cruise RMS max (m)	Max leaders at once	Leader sequence	Landed	Sim speed (× real time)	Heartbeat (B)	Radio load (kbit/s)
10	2.35 (2.15–2.40)	4.80 (4.65–4.90)	9.15	1.14	1	1→2	3/3	150.4 (147.8–153.5)	45	18.0
50	2.55 (2.55–2.75)	5.70 (5.40–6.30)	8.68	1.11	1	1→2	3/3	10.3 (9.7–10.9)	50	100.0
100	2.75 (2.50–2.95)	9.20 (8.95–9.30)	8.14	1.05	1	1→2	3/3	2.6	56	224.0

All 33 runs completed with every drone landed and exactly one leader at the end; in the harder-radio runs the leader sequence was always 1 → 2 and there were never two leaders at once. Closest approach never went below 8.14 m (limit 5 m). The worst new-leader time was 2.95 s ($N = 100$, harder radio) against the Phase 4 target of 3.0 s median / 4.0 s worst.

What changed to make 100 drones possible

The heartbeat's member list was a 64-bit mask, so the configuration refused more than 63 drones (`ConfigError: drone IDs must be within 1..63`, checked before the change). The mask is now sent with only as many bytes as the highest drone ID needs; IDs run 1..250, the PX4 `MAV_SYS_ID` range (`src/modules/mavlink/mavlink_params.yaml` in PX4 v1.18.0-rc1: min 1, max 250). Heartbeat size: 44 B for up to 7 drones, 45 B for 10, 50 B for 50, 56 B for 100, 75 B for 250 (`tests/test_heartbeat.py`).

Honest limits

- **Simplified physics.** This is the point-mass simulator, not PX4. PX4 SIH on this laptop stays at 10 drones (Phase 1: 77 % CPU at 10; PX4's default port plan in `px4-rc.mavlink` covers 10 instances).
- **Simulator cost grows with N^2 .** Every drone checks every other drone: 162× real time at 10 drones, 2.9× at 100 (clean runs).
- **Radio load grows with N.** Heartbeats go to every drone, 5 per second: 18 kbit/s at 10 drones, 224 kbit/s at 100 (payload only, before framing). ArduPilot's SiK radio documentation gives a default air rate of 64 kbit/s, a maximum of 250, and usable throughput of about 90 % of the air rate (half with error correction): <https://ardupilot.org/copter/docs/common-3dr-radio-advanced-configuration-and-technical-information.html>
- **One V for 100 drones is impractical.** Each arm is about 490 m long and the last drones land 490 m from the target. Big swarms should fly as squads (for example ten groups of ten, each with its own leader).
- The formation takes longer to recover at 100 drones (8.75 s clean, 9.2 s harder radio) because every drone on one arm shifts one slot when the leader's successor moves to the tip; still inside the 15 s Phase 4 target.

Appendix G - Project specification

DOCS/SPECIFICATION.MD

Swarm Failover — Project specification

The specification this project was built against: goals, hard rules, phases and acceptance criteria. Section and rule numbers quoted in the code and the reports (for example §6 or rule 3a) refer to this document.

1. Goal

Build, on this laptop only, a simulated swarm of PX4 quadrotors that flies in V formation from home to a goal waypoint. One drone is MASTER. If the master fails (killed, link lost, or low battery), the lowest-ID alive drone becomes master within seconds and the swarm continues to the goal.

The architecture must later run unchanged against real Pixhawk 6C drones: switching a drone from simulated to real must only require changing its connection URL in config.

In scope: waypoint navigation, formation, leader election / failover, inter-drone separation, geofence, fault injection, metrics. **Out of scope:** payloads, targeting, or any logic that engages objects.

2. Environment (VERIFY in Phase 0 — do not assume)

- Laptop: Lenovo IdeaPad 5, Intel i3 11th gen, 8 GB RAM, no GPU
- OS: Ubuntu 24.04, ROS 2 (expected: Jazzy), Python 3.12
- Simulator: PX4 SIH running as SITL (headless, no Gazebo). Build target: `px4_sitl_sih`
- Drone link: MAVROS, one instance per drone (NOT uXRCE-DDS), because real drones will use MAVLink over low-bandwidth telemetry radios.
- User is on mobile most of the time and is notified via ntfy (see §4).

3. Hard rules (apply to every phase)

- Verify before use.** Every PX4 parameter, make target, MAVROS topic/service, ROS 2 command, and package name must be confirmed from official docs (docs.px4.io, docs.ros.org, github.com/mavlink/mavros) OR from the installed system (`ros2 topic list`, `ros2 interface show`, `param show`, reading source). Record the source (URL or command) in the phase report. If you cannot verify something, say so — never guess.
- Evidence or it didn't happen.** Never report success without command output, a test result, or a metrics file. Put raw logs in `reports/logs/phase_N/`.
- Cross-check at phase end.** Before reporting a phase done: a. Kill all sim processes (`kill -f px4`; `kill -f mavros`), open a fresh shell, re-run the phase's acceptance tests from scratch. b. Re-read this file's acceptance criteria line by line and mark each PASS/FAIL with evidence. c. Confirm every number in the report matches the raw logs.
- Phase gates.** After each phase: write `reports/PHASE_N.md`, commit, tag `phase-N`, send the notification, then STOP and wait for the user to type `continue`. Do not start the next phase on your own.
- Stuck rule.** If the same error survives 3 fix attempts, stop, notify (priority high), and write what you tried and what you suspect in the report.
- Resource guard.** Build with `-j2` max. Check `free -m` before launching sims; if available memory < 800 MB, reduce drone count and report it. Kill orphan `px4/mavros` processes between runs.
- Safety.** Simulation only. Never open serial devices (`/dev/tty*`), never connect to a real flight controller, never flash firmware.
- Ask first** before: any `sudo`, `apt/pip` installs, deleting files outside this repo, or changing an architecture decision in this file (e.g., replacing MAVROS).
- Code standards.** Python 3.12, rclpy, type hints, all tunables in `config/swarm.yaml` (no hardcoded ports/IDs), pure-Python logic modules (election, formation, geometry) importable and testable WITHOUT ROS.

4. Notifications (user's phone via ntfy)

Send with: `scripts/notify.sh "<title>" "<message>" [priority]` - Title must be plain ASCII (it is an HTTP header). Emojis only in the message. - Phase done: `scripts/notify.sh "Phase N PASSED" "<1-line key result>". Waiting for continue.` - Phase failed / stuck: same format, title `Phase N FAILED`, priority `high`. - Long run (> 20 min) started: one message with the ETA. - In Phase 0, send a test message and ask the user to confirm it arrived.

5. Mission defaults (`config/swarm.yaml`)

Setting	Value
Drones (target)	10 (start with 3, scale in Phase 1)
Formation	V, 10 m spacing between neighbors
Cruise altitude / speed	30 m AGL / 5 m/s
Goal	single point ~1 km from home, loaded into every drone before takeoff
Heartbeat	5 Hz, each drone broadcasts: id, role, term, position, velocity, battery, timestamp
Master dead after	1.5 s without its heartbeat
Election rule	lowest alive ID wins; <code>term</code> counter resolves conflicts (see §6)
Low-battery handover	master hands over voluntarily at 30 %, then leaves formation and returns home
Min separation (hard)	5 m between any two drones
Geofence	1.5 km radius from home, 50 m max altitude
Setpoint stream	velocity setpoints at 20 Hz (PX4 drops offboard if the stream falls below 2 Hz — verify in docs)

6. Architecture

- **One `swarm_agent` node per drone** (namespace `/uavN`) talking to its own MAVROS instance. There is NO central controller node — each agent decides its own role, so the same code can later run onboard each drone.
- Central tools allowed only for: launching, fault injection, link emulation, logging.
- **Heartbeat transport goes through a `link_emulator` node**: agents publish to `/uavN/hb_out`, the emulator forwards to every other `/uavM/hb_in` with configurable loss/latency per link. This is how "link lost" faults and radio-like conditions are simulated.
- **Election**: each agent keeps `term`. A drone that becomes master increments `term`. On receiving a master heartbeat with a higher `term` (or equal `term` and lower ID), a master steps down. After a partition heals there must be exactly one master.
- **Formation**: follower slot = rank among alive drones sorted by ID. Slot offset is rotated into the master's heading. Follower command = master velocity (feedforward)
- P-control on position error; saturated; add a repulsion term when any neighbor is closer than $1.5 \times$ min separation.
- **Frames — critical**: every drone's MAVROS local frame has its own origin. NEVER compare drones' local positions directly. Convert each drone's global position (lat/lon/alt) to one shared ENU frame anchored at `config.origin`. Unit-test this.
- **No master heard and no election result yet**: hold position (hover).
- **At goal**: swarm hovers 10 s, then all drones land.

7. Phases

Phase 0 — Environment audit and setup

Tasks: - Record OS, `$ROS_DISTRO`, RAM, CPU cores, free disk (need ≥ 15 GB), Python version. - Test `ntfy` (`scripts/notify.sh "Phase 0" "Test message"`) and ask user to confirm. - Install (ask first): PX4-Autopilot toolchain for Ubuntu 24.04 per docs.px4.io, MAVROS for the installed ROS 2 distro, and the GeographicLib datasets MAVROS requires. - Build `px4_sitl_sih` for quadrotor (-j2). - Create repo skeleton: `config/`, `src/swarm_agent/`, `tests/`, `scripts/`, `reports/`. Acceptance: - One SIH drone + one MAVROS instance: script arms, takes off to 10 m, hovers 20 s, lands. Evidence: log with altitude over time. - Peak RAM for 1 drone (PX4 + MAVROS) measured and recorded.

Phase 1 — Multi-drone simulation scaling

Tasks: - Launch script for N SIH instances + N MAVROS instances with unique ports/sysids (verify the multi-instance method in PX4 docs). - Verify spawn positions. If instances spawn at the same point, give each a distinct home position using the documented method, and cite it. - Scale $3 \rightarrow 5 \rightarrow 10$. At each step measure RAM and CPU. Acceptance: - All N drones arm, take off to 10 m, hover 30 s, and land, with no crashes. - Report: max stable N on this laptop with RAM/CPU numbers. If $N < 10$, state it plainly and propose options (e.g., lighter MAVLink bridge) — do not switch without approval.

Phase 2 — Agent logic (pure Python, no ROS) + unit tests

Tasks: - Modules: `geometry` (global $\rightarrow$ shared ENU), `election` (state machine + `term`), `formation` (V slots, rotation, rank reassignment), `safety` (separation, geofence). - Pure-Python swarm simulator (point-mass, no PX4) to exercise election + formation fast. Acceptance: - `pytest` all green; coverage $\geq 90\%$ on election and formation modules. - 1,000 randomized pure-Python runs (random kills, random link drops, partitions): exactly one master after convergence in 100 % of runs; median convergence time reported.

Phase 3 — ROS 2 integration: formation flight, no faults

Tasks: - `swarm_agent` node wrapping the Phase 2 modules; `link_emulator` node (loss = 0); launch file; logger node writing CSV (positions, roles, terms, timestamps). - Metrics script: formation RMS error, min pairwise separation, time to goal. Acceptance (3 runs at max stable N): - All drones reach goal and land. - Formation RMS error < 2 m during cruise. - Min separation ≥ 5 m at all times. - Plot of trajectories saved to `reports/`.

Phase 4 — Failover under faults

Faults (fault injector node/script): - F1 master killed (kill its PX4 + MAVROS processes) - F2 master link lost (emulator drops 100 % of master heartbeats) - F3 master low battery (use PX4's simulated battery drain if verifiable in docs; otherwise inject battery value at the agent level and say so in the report) - F4 follower killed (no election expected; slots reassign) - F5 partition then heal (two groups, then reconnect) Acceptance (10 trials per fault, random fault time during cruise): - F1/F2: new master heartbeat within 3.0 s median (4.0 s worst case) of failure. - F3: planned handover within 1.0 s of trigger; old master leaves and returns home. - F4: no master change; formation recovers. - F5: single master within 3.0 s after heal. - All: formation RMS back under 2 m within 15 s; min separation never < 5 m; goal reached in $\geq 9/10$ trials per fault. - Report table: per fault — median/worst handover time, recovery time, min separation, success rate.

Phase 5 — Radio realism (telemetry-like links)

Tasks: - Sweep link emulator: latency {50, 150, 300} ms $\times$ loss {0, 10, 30} %. - Measure: false failovers (master change without a real fault), handover time, formation error. Acceptance: - Results table + plot. Identify the worst condition where Phase 4 criteria still hold. - Zero false failovers at $\leq 10\%$ loss; if not, report and propose timeout/heartbeat changes.

Phase 6 — Mixed-reality readiness (still software only)

Tasks: - `config/profiles/`: `sim.yaml` and `mixed.yaml` (example: drone 1 real via serial URL placeholder, drones 2–N SIH). Do NOT open the serial port — validate config only. - Hardware stand-in test: run one SIH drone through a UDP proxy that limits bandwidth and adds latency/loss like a telemetry radio; the swarm must still pass Phase 4 F1 and F2. - Write `FLIGHT_TEST_PLAN.md`: 1 real + N virtual drones, geofence, RC override on every real drone, kill-switch procedure, go/no-go checklist, abort criteria. - Write `README.md`: setup, how to run each phase, results summary, known limitations. Acceptance: - Stand-in test passes F1 and F2 criteria. - Everything reproducible from a fresh clone with the README. - FINAL STOP. Notify: "Software phase complete. Real-drone work needs user review."

8. Phase report template (`reports/PHASE_N.md`)

```
# Phase N - <name>
Status: PASSED / FAILED
## Acceptance criteria
- [PASS/FAIL] <criterion> - evidence: <file/command/number>
## Key numbers
## Sources verified (URLs / commands)
## Problems and fixes
## Known limitations / honest caveats
## Next phase: what is needed from the user (if anything)
```